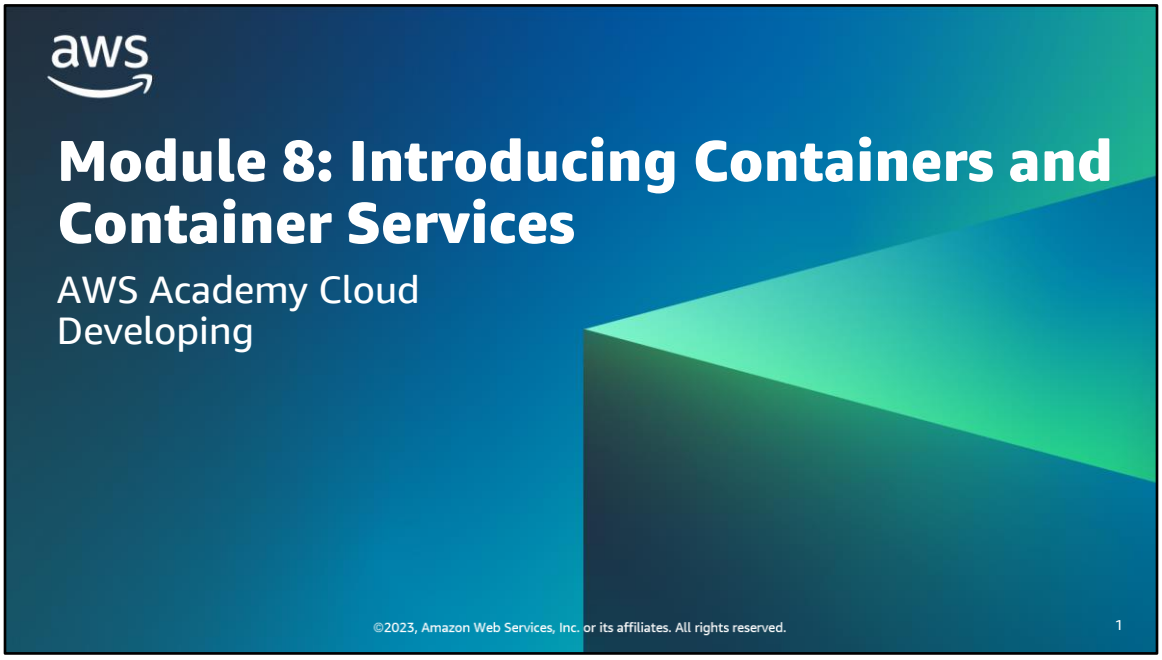
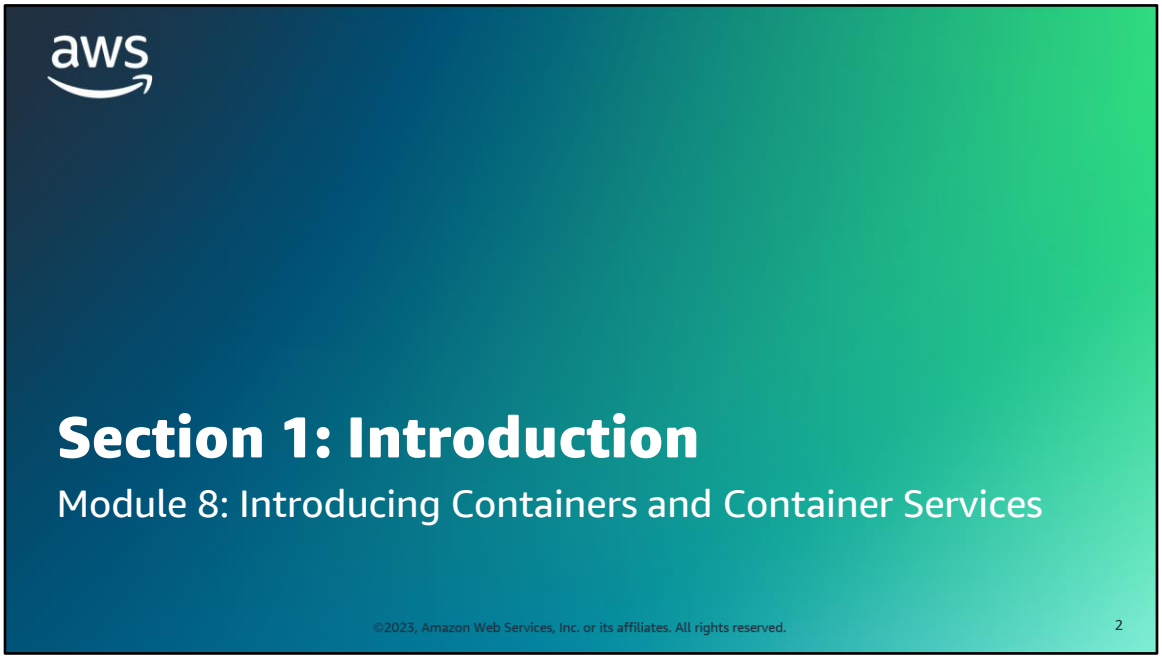
AWS Academy Cloud Developing
Module 08 Student Guide
Version 2.0.6

200-ACCDEV-20-EN-SG

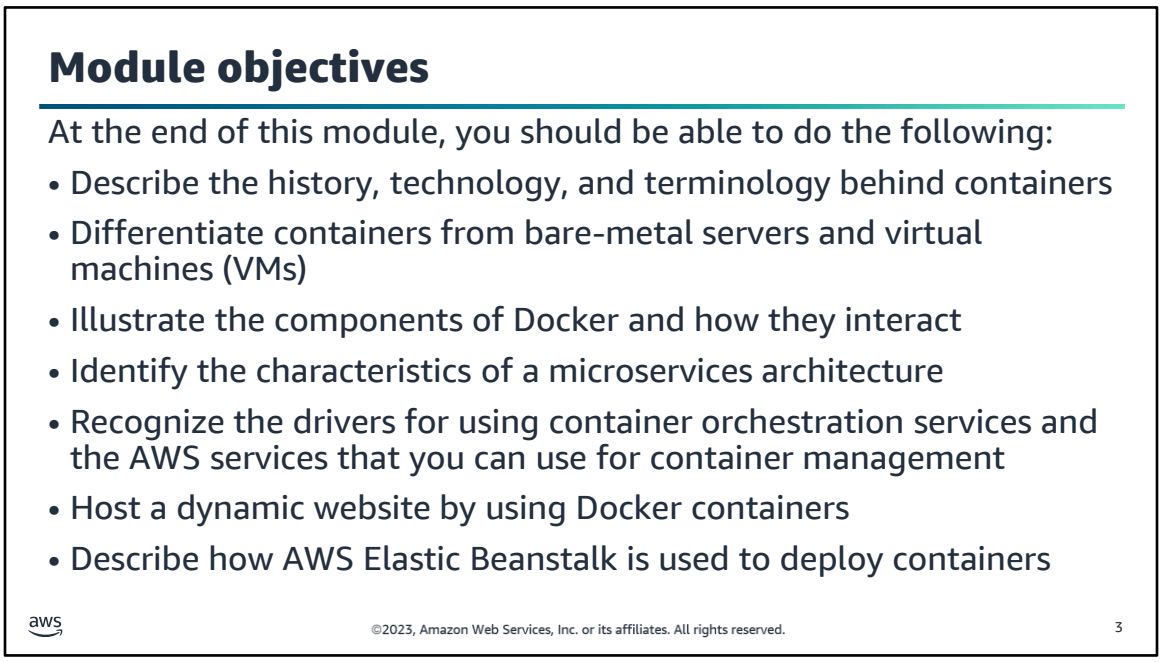
Contents



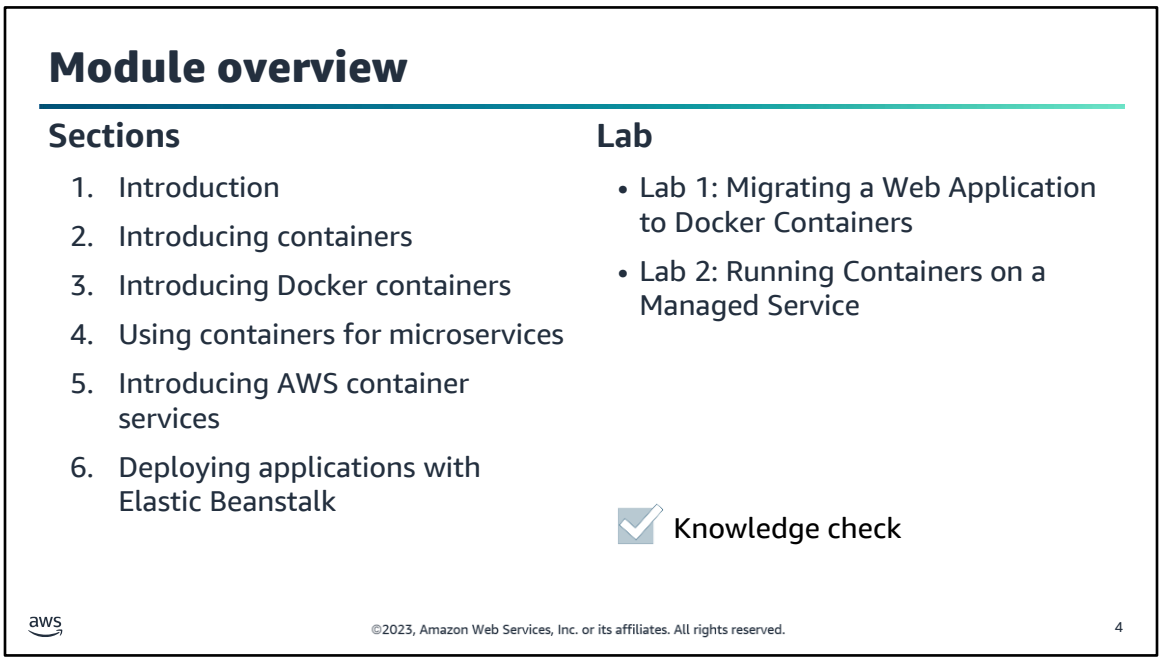
Welcome to Module 8: Introducing Containers and Container Services.



Section 1: Introduction.



At the end of this module, you should be able to do the following:
- Describe the history, technology, and terminology behind containers
- Differentiate containers from bare-metal servers and virtual machines (VMs)
- Illustrate the components of Docker and how they interact
- Identify the characteristics of a microservices architecture
- Recognize the drivers for using container orchestration services and the AWS services that you can use for container management
- Host a dynamic website by using Docker containers
- Describe how AWS Elastic Beanstalk is used to deploy containers



This module includes the following sections:

1. Introduction
2. Introducing containers
3. Introducing Docker containers
4. Using containers for microservices
5. Introducing AWS container services
6. Deploying applications with Elastic Beanstalk

This module also includes two labs where you will work with Docker containers.

Finally, you will complete a knowledge check to test your understanding of key concepts covered in this module.

Café business requirement

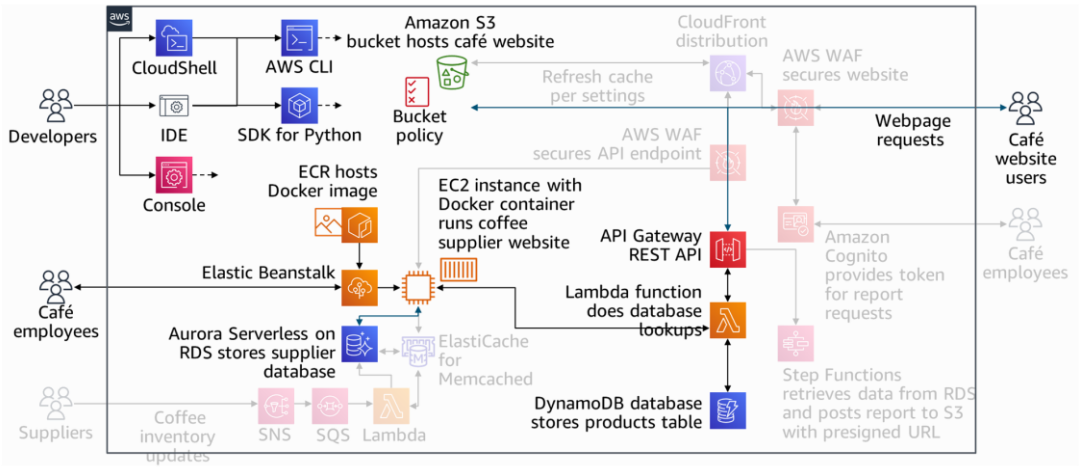
Frank and Martha recently acquired a coffee bean supplier, and they would like to include the supplier's inventory tracking system into the café's application infrastructure. Sofía is thinking about migrating the application database to containers to complete the integration.

aws

5

Frank and Martha recently acquired a coffee bean supplier, and they would like to include the supplier's inventory tracking system into the café's application infrastructure. Sofía is thinking about migrating the application database to containers to complete the integration. The supplier's inventory tracking application runs on an AWS account. Sofía has been asked to learn how the application works and then create a plan to integrate it into the café's existing application infrastructure.

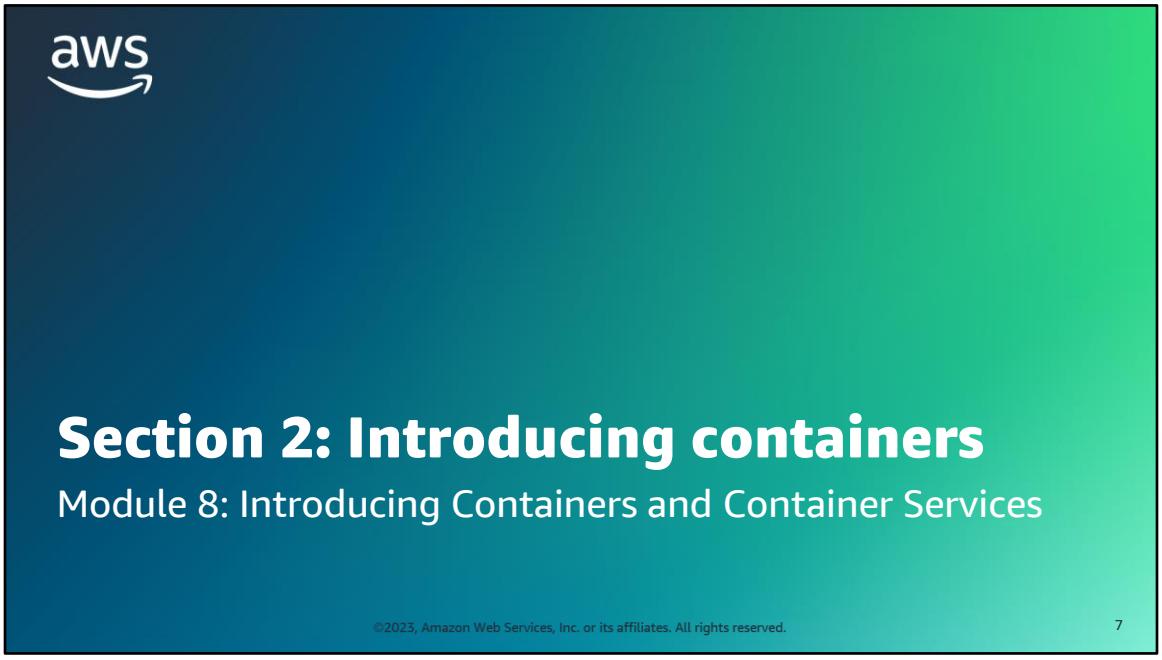
Containers as part of developing a cloud application



 6

The diagram on this slide gives an overview of the application that you will build through the labs in this course. The highlighted portions are relevant to this module.

As highlighted in the diagram, you will first migrate the coffee suppliers application to run on containers. Then, you will deploy the database tier with Amazon Aurora Serverless and deploy the web tier with Elastic Beanstalk.



Section 2: Introducing containers.

Shipping containers

Before shipping containers

- Goods were shipped in a variety of vessels with no standardized weight, shape, or size.
- Transporting goods was slow, inefficient, and costly.

After shipping containers

- Uniformly sized shipping containers simplified loading, unloading, storing, and transferring between transport types.
- Abstraction of shipment details improved efficiency, increased productivity, and reduced costs.

aws

8

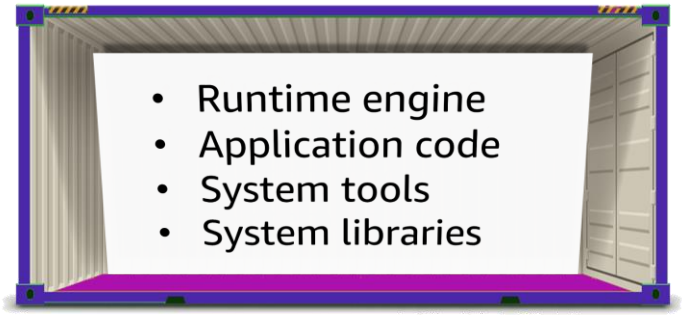
In the physical world, a *container* is a standardized unit of storage. *Container* sounds like a generic term, but its importance is clearer if you look at how containers changed the shipping industry.

Before the introduction of modern shipping containers, transporting goods from one point to another was a challenge. Goods were shipped in sacks, crates, cartons, drums, casks, barrels, and boxes of various weights, shapes, and sizes. They often were loaded by hand into whatever vessel was carrying them. The vessel wouldn't know how much cargo it could take until all of the cargo was loaded. Transporting goods this way was slow, inefficient, and costly.

Shipping containers revolutionized the shipping industry. Their uniform size made it much more efficient to load, unload, and stack them. Containers could also be easily moved between ships, trucks, and railroad cars. In this way, containers improved efficiency, increased productivity, and reduced costs.

This is a great example of using abstraction to increase agility. Abstraction, in this sense, means that the people who are involved in shipping are further removed from the details of the transaction. This abstraction applies to the people who send the contents and those who actually facilitate the shipping itself. They don't need to worry about whether items will fit in a given container or about the fit of the container onto various vessels. Abstraction is a way of hiding the working details of a subsystem. It separates concerns to facilitate interoperability and platform independence.

A container is a standardized unit of software

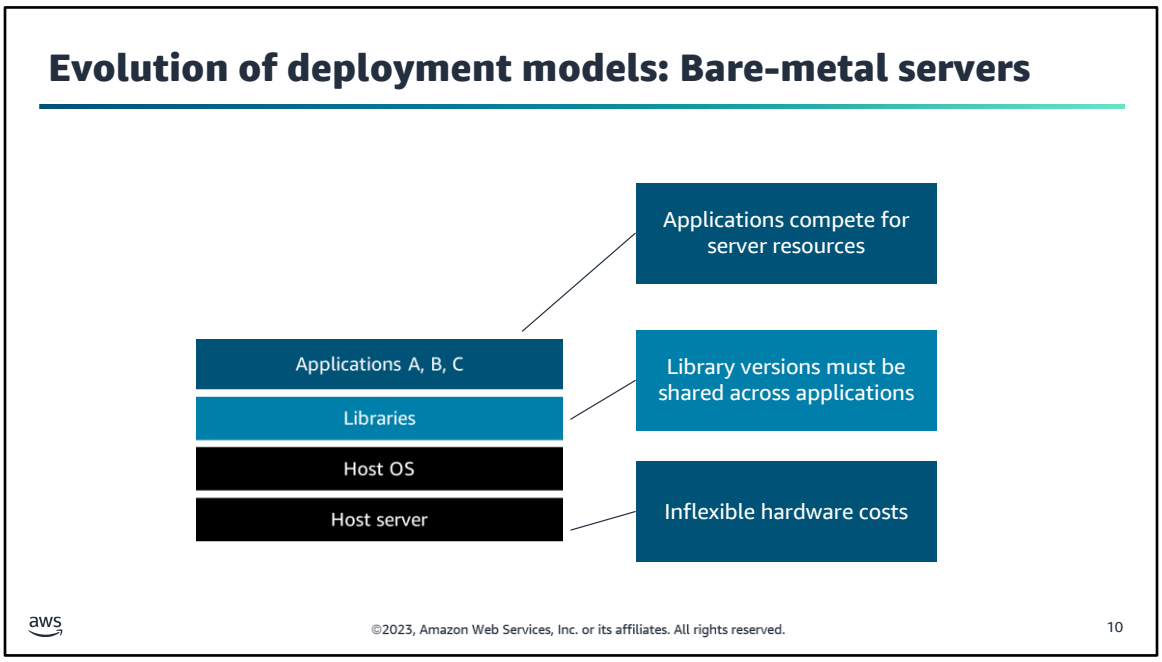


 9

This concept can move from the physical world to the virtual world of computing platforms. Virtually, a *container* is a standardized unit of software designed to run quickly and reliably on any computing environment that runs the containerization platform.

Containers provide operating system (OS) virtualization so that you can run an application and its dependencies in resource-isolated processes. A container is a lightweight, standalone software package that contains everything that a software application needs to run. For example, it can contain the application code, runtime engine, system tools, system libraries, and settings. Containers can help ensure that applications deploy quickly, reliably, and consistently regardless of the deployment environment.

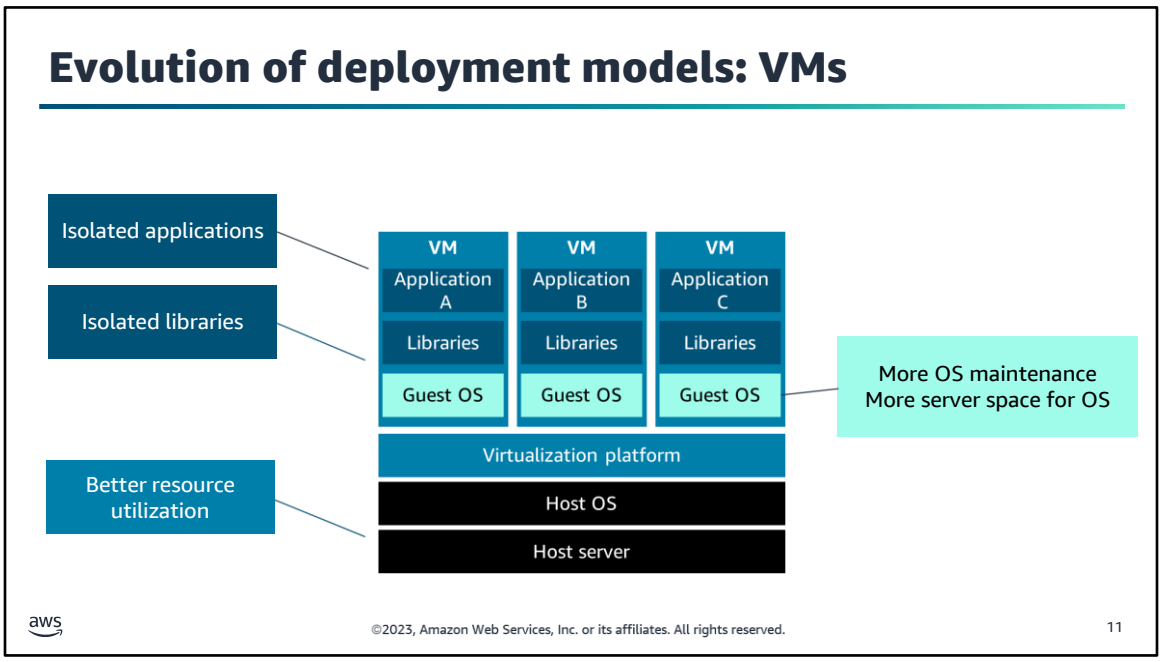
A single server can host several containers that all share the underlying host system's OS kernel. These containers might be services that are part of a larger enterprise application, or they might be separate applications that run in their isolated environment.



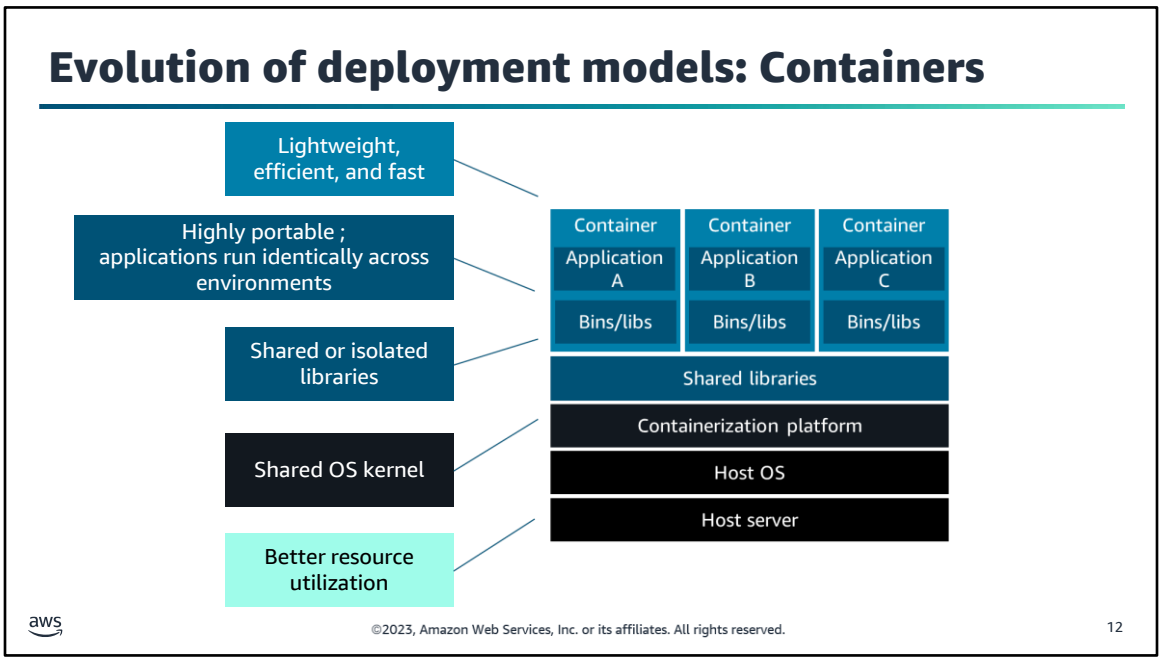
Increasing levels of abstraction are often associated with technical maturity. The move toward containers is part of the evolution of deployment models.

In earlier phases, you had *bare-metal servers.* You had to build the architectural layers, such as the infrastructure and application software layers. For example, you would install an OS on top of your server hardware. Then, you would install any shared libraries on the OS, and then install the applications that use those libraries. This architecture was inefficient.

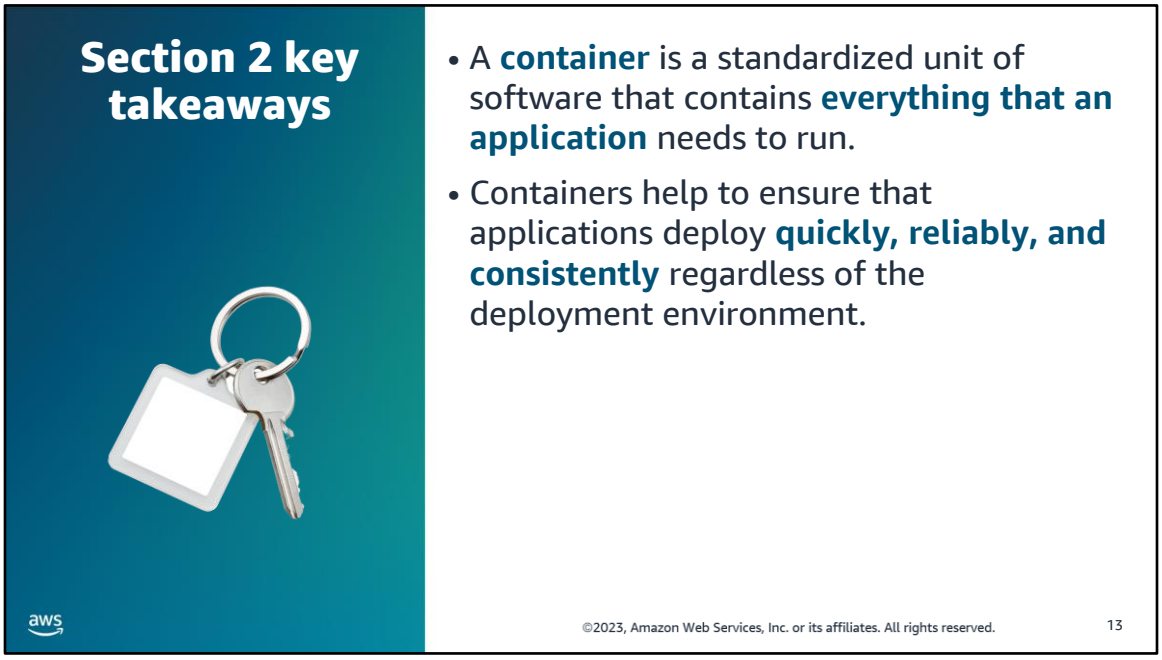
Your hardware costs are the same whether you are running at 0 percent utilization or 100 percent utilization. All of your applications must compete for the same resources, and you must keep the versions of your libraries in sync with all your applications. If one application requires an updated version of a library that is incompatible with other applications running on that host, then you run into problems.



These shortcomings led to the use of a virtualization platform over the host OS. Now, you have isolated applications and their libraries, which have their own full OS inside a VM. This arrangement improves utilization because you can add more VMs to run on top of the existing hardware, which greatly reduces your physical footprint. The downside to VMs is that the virtualization layer is heavy. In this example, you now have three operating systems on the physical host server, instead of one. That means more patching, more updates, and significantly more space that is taken up on the physical host. It also can cause significant redundancy: you have installed potentially the same OS three times, and potentially the same library three times.

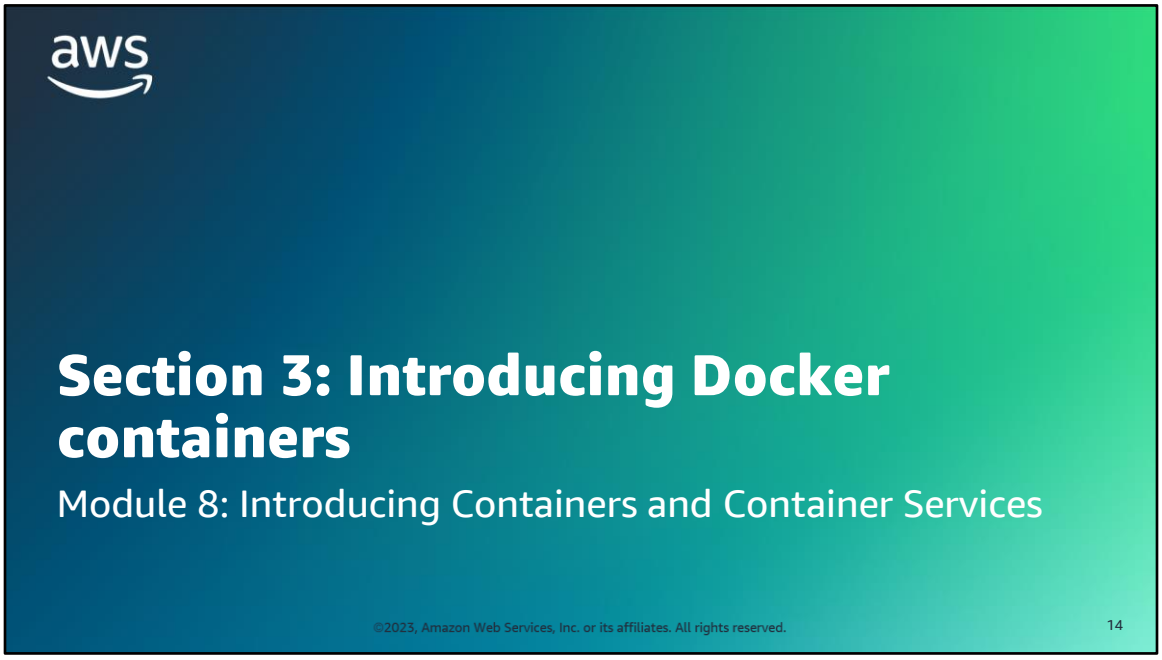


Containers improved upon the idea of virtualization. The container runtime shares the host operating system's kernel. You can use this arrangement to your advantage to create container images by using file system layers. Containers are lightweight, efficient, and fast. They can be started up and shut down faster than VMs, which means better utilization of the underlying hardware. You can share libraries when needed, but you can also have library isolation for your applications. Containers are also highly portable. Because containers isolate software from other layers, their code runs identically across different environments—from developing and staging all the way to production.

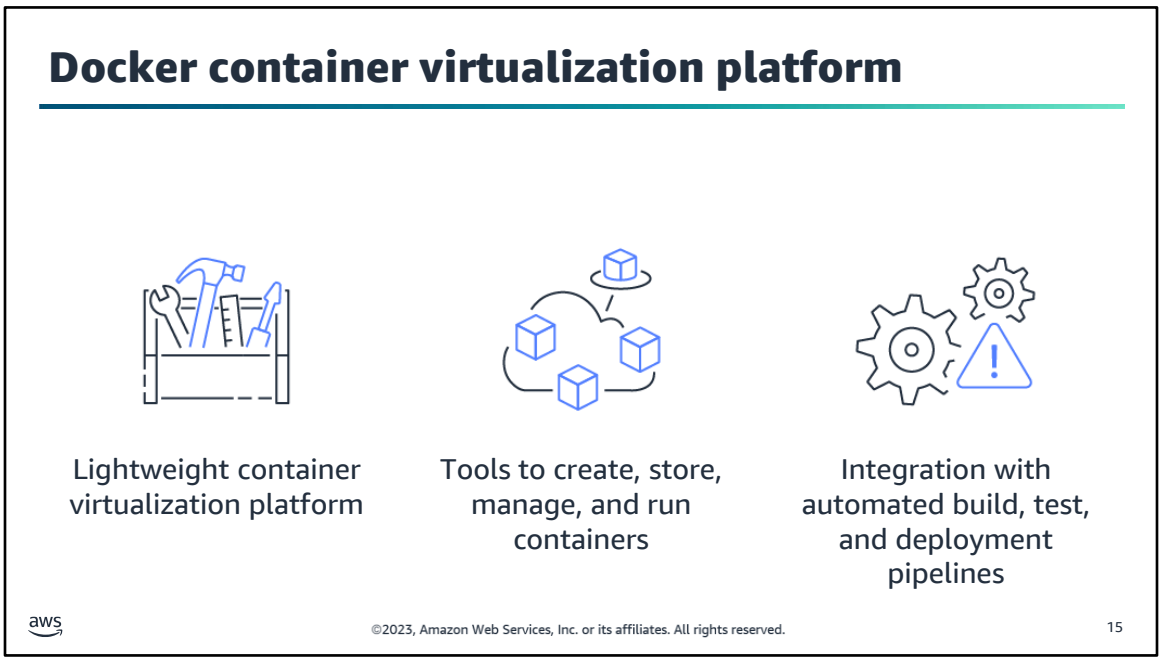


The following are the key takeaways from this section of the module:

- A container is a standardized unit of software that contains everything that an application needs to run.
- Containers help to ensure that applications deploy quickly, reliably, and consistently regardless of the deployment environment.



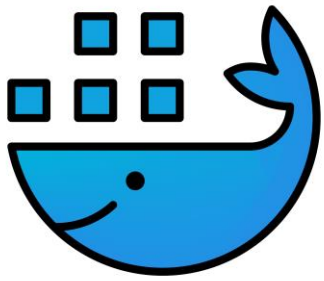
Section 3: Introducing Docker containers.



Containers have become popular due to the rise of Docker as a container virtualization platform. This lightweight platform provides tooling to create, store, manage, and run containers. It is easy to integrate with automated build, test, and deployment pipelines.

Docker container benefits

- **Portable** runtime application environment
- Application and dependencies can be packaged in a **single, immutable artifact**
- Ability to run different application versions with different dependencies **simultaneously**
- **Faster** development and deployment cycles
- Better **resource utilization and efficiency**



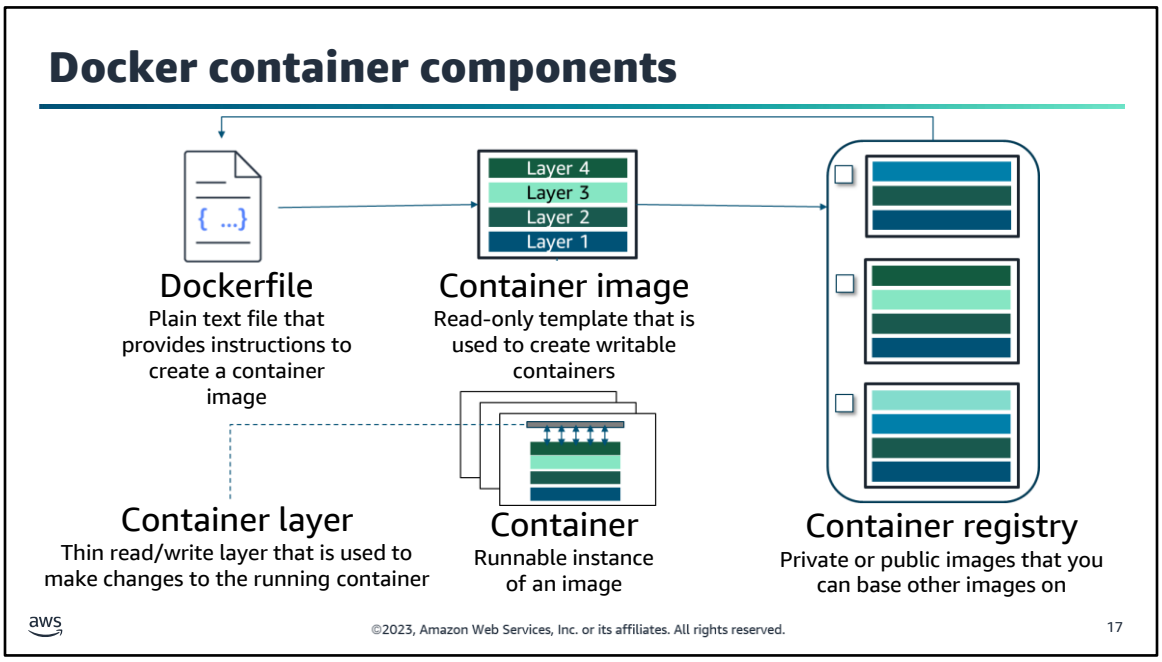
aws

16

The following list summarizes some of the important benefits of Docker containers:

- Docker is a portable runtime application environment.
- You can package an application and its dependencies into a single, immutable artifact that is called an *image*.
- After you create a container image, it can go anywhere that Docker is supported.
- You can run different application versions with different dependencies simultaneously.

These benefits lead to much faster development and deployment cycles, and better resource utilization and efficiency. All of these abilities are related to agility.



Docker containers are created from read-only templates, which are called *container images*. Images are immutable and highly portable. You can port an image to any environment that supports Docker. Images are composed of layers.

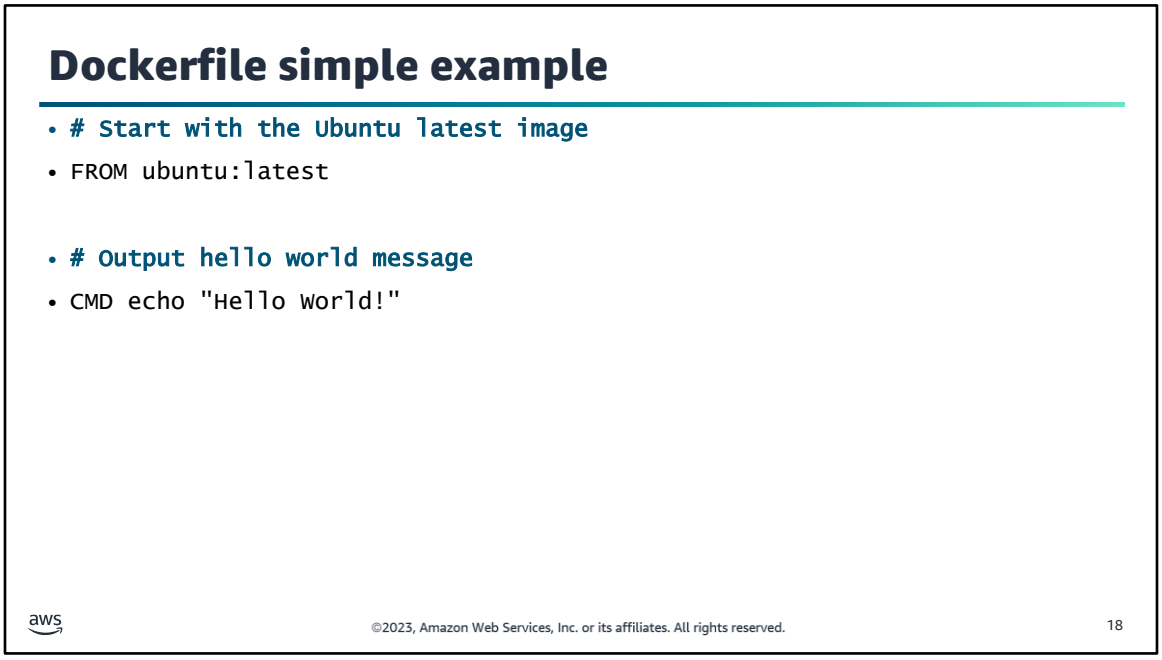
Images are built from a *Dockerfile*, which is a plain text file that specifies all of the components that are included in the container. Instructions in the Dockerfile create layers in the container image.

You can create images from scratch, or you can use images that others created and published to a public or private container registry.

An image is usually based on another image, with some customization. For example, you might build an image that's based on the Ubuntu Linux image in the registry. The image could also install a web server and your application, in addition to the essential configuration details to make your application run.

A *container* is a runnable instance of an image. You can create one container or several containers based on that image.

Each container has a thin, read/write layer on top of the existing image when it is instantiated. This architecture is what makes the actual process of instantiating the containers fast. Most of the actual work is read-only because of the file system layers. The read/write layer of the container enables your applications to function properly while they are running, but it's not designed for long-term data storage. Persistent data should be stored in a volume somewhere. Consider a container as a discrete compute unit, not a storage unit.



To build your own image, you create a Dockerfile by using a simple syntax to define how to create the image and run it. Each instruction in a Dockerfile creates a read-only layer in the image.

In this simple example, you start with the Ubuntu latest image that is already created for you, and hosted on Docker Hub or some other site. The only thing that you do is add a command to echo the message *Hello World!* after the container is instantiated.

Dockerfile example: Start a Java application

```
# Start with open JDK version 8 image
FROM openjdk:8

# Copy the .jar file that contains your code from your system to the
container
COPY /hello.jar /usr/src/hello.jar

# Call Java to run your code
CMD java –cp /usr/src/hello.jar
Org.example.App
```


19

This is a slightly more involved example, in which you want to run a Java application. You start with the open Java Development Kit (JDK) version 8 image. You copy the .jar file that contains your code from your system to the container and then call Java to run your code. When this container is instantiated, it runs the Java application.

Dockerfile example: Common tasks

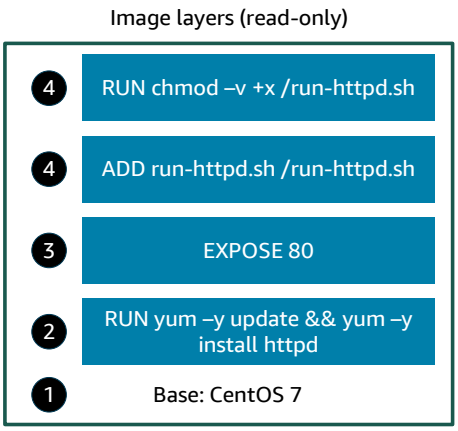
```
# Start with CentOS 7 image
FROM centos:7

# Update the OS and install Apache
RUN yum –y update && yum –y install httpd

# Expose port 80—the port that the web server “listens to”
EXPOSE Port 80

# Copy shell script and give it run permissions
ADD run-httpd.sh /run-httpd.sh
RUN chmod –v +x /run-httpd.sh

# Run shell script
CMD ["/run-httpd.sh"]
```

 20

This is a more real-world example of a Dockerfile. You start with the CentOS 7 image. Next, you update the OS and install Apache. Then, you expose Port 80. Finally, you copy the shell script for your application and give it run permissions. After the container is instantiated, the command will run the shell script.

Each line of the Dockerfile adds a layer

```
# 1 Start with CentOS 7 image
FROM centos:7

# 2 Update the OS and install Apache
RUN yum –y update && yum –y install httpd

# 3 Expose port 80
EXPOSE Port 80

# 4 Copy shell script and give it run
permissions
ADD run-httpd.sh /run-httpd.sh
RUN chmod –v +x /run-httpd.sh

CMD ["/run-httpd.sh"]
```



aws

21

This example shows what the previous example looks like in terms of layers. Each instruction in the Dockerfile creates a layer.

- The first layer, starting from the bottom of the diagram, is the base layer
- The next layer includes the software update and the Apache installation.
- The next layer opens and exposes port 80.
- The layer after that includes the command to copy the shell script.
- Finally, the last layer makes the shell script run.

These layers are all read-only, which makes the container image an immutable object.

If you change the Dockerfile and rebuild the image, only the layers that have changed are rebuilt. This feature is part of what makes container images so lightweight, small, and fast compared to other virtualization technologies.

Docker CLI commands

Command	Description
`docker build`	Build an image from a Dockerfile.
`docker images`	List images on the Docker host.
`docker run`	Launch a container from an image.
`docker ps`	List the running containers.
`docker stop`	Stop a running container.
`docker start`	Start a container.
`docker push`	Push the image to a registry.
`docker tag`	Tag an image.

Command	Description
`docker logs`	View container log output.
`docker port`	List container port mappings.
`docker inspect`	Inspect container information.
`docker exec`	Run a command in a container.
`docker rm`	Remove one or more containers.
`docker rmi`	Remove one or more images from the host.
`docker update`	Dynamically update the container configuration.
`docker commit`	Create a new image from a container's changes.

aws

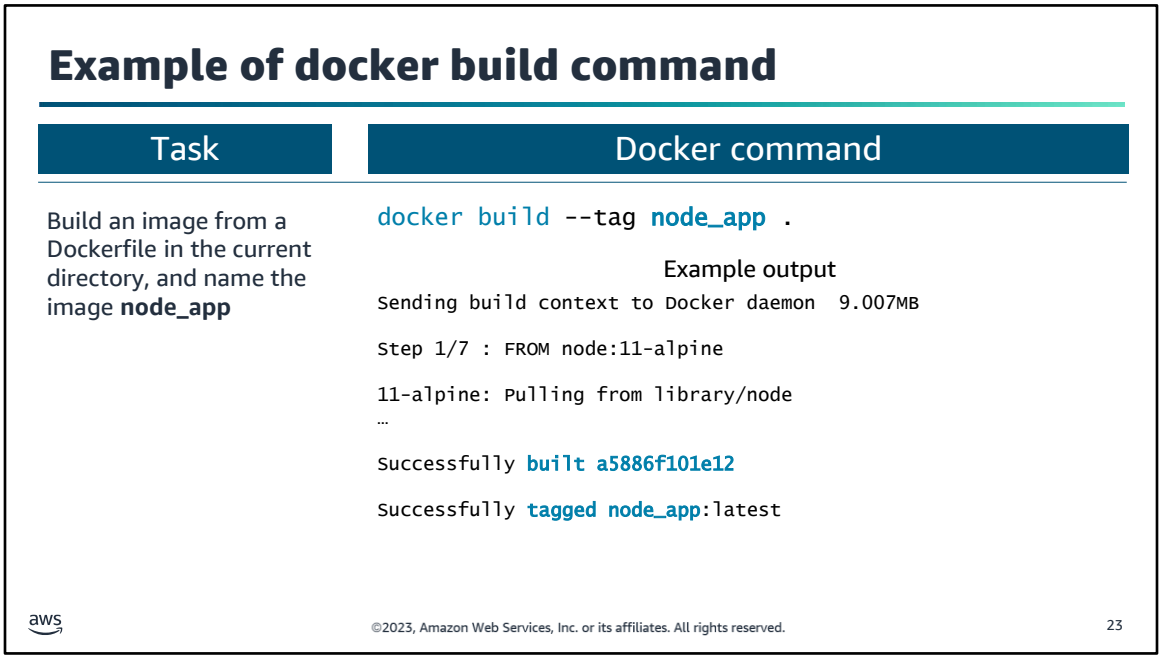
22

You can run Docker command line interface (CLI) commands from a Bash terminal to manage your Docker images and containers.

For example, you can build an image from a Dockerfile by running docker build. You can then verify your image by running docker images. To launch a container from the image, run docker run. To verify that the container is running, enter *docker ps*.

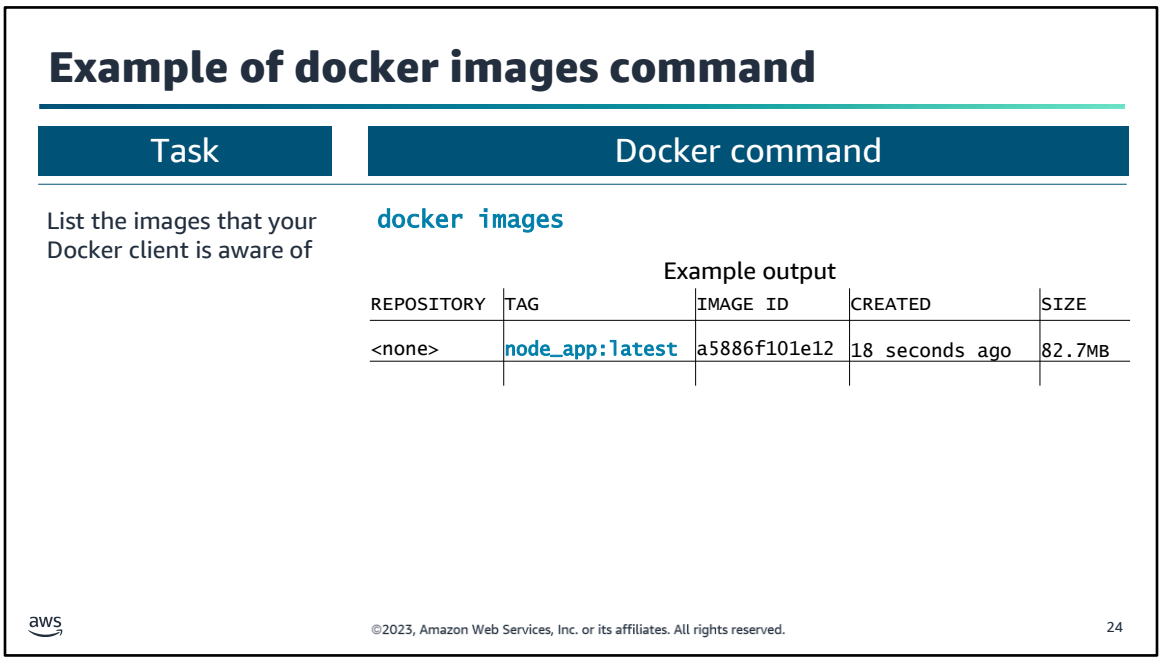
After you launch a container, you can interact with it in a variety of ways. To open a Bash prompt on your running container, use docker exec. You can also stop a running container (docker stop) and start it again (docker start). To view your container logs, enter *docker logs*. To list the port mappings for your container, run docker port. Run docker tag to tag your image and use docker push to push it to a registry.

For more information about Docker CLI commands, see https://docs.docker.com/engine/reference/commandline/cli/.



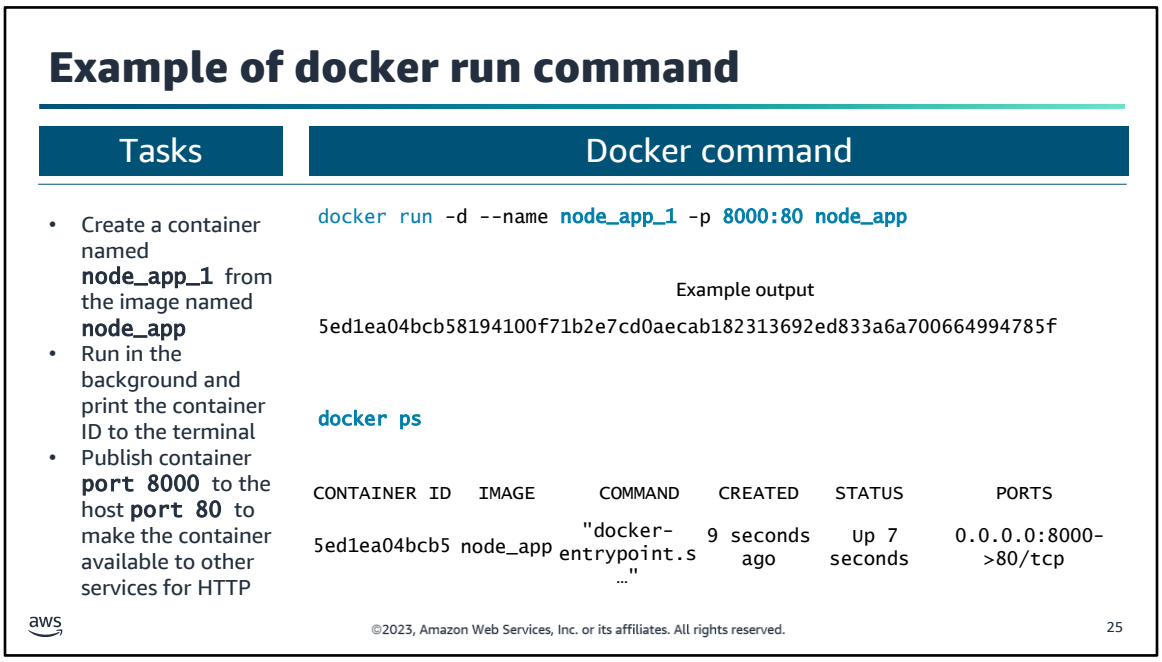
This example runs the `docker build` command to create an image from a Dockerfile in the current directory. The `tag` option names the new image `node_app`.

The actual output would show the progress of each instruction in the Dockerfile (steps 1 through 7).



This example runs the docker images command to list the images that your Docker client is aware of. The actual output might include other images, but the example output highlights the image that was created with the build command in the previous slide.

The actual output would show the progress of each instruction in the Dockerfile.



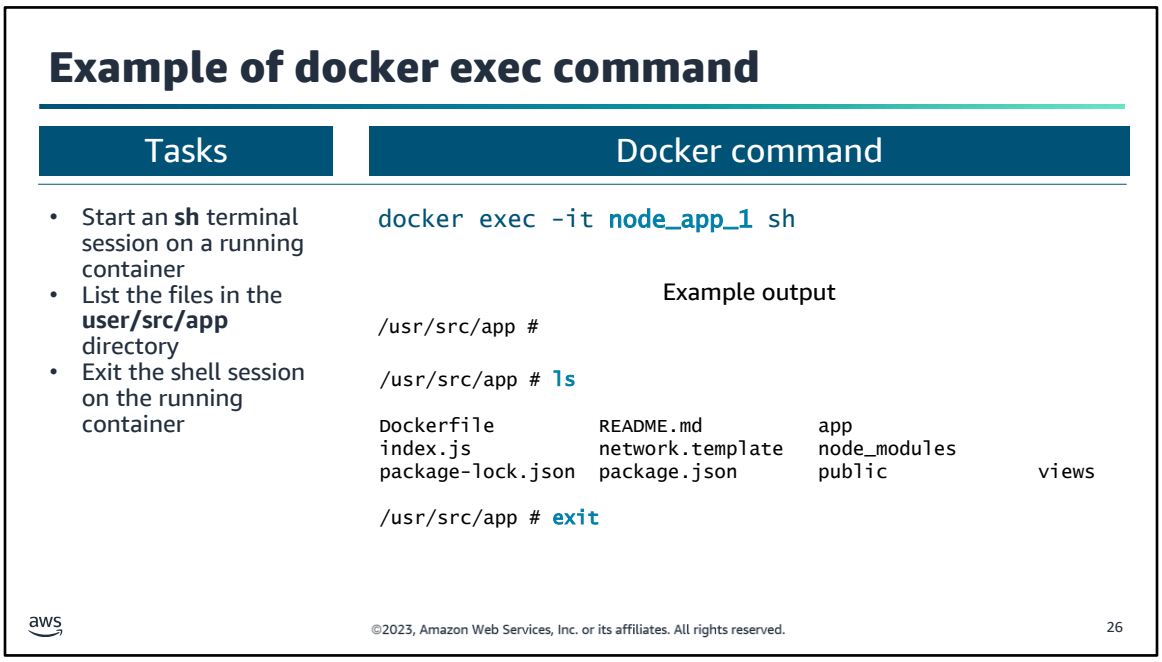
This example runs the docker run command to create a container from the node_app image.

The -d (or –detach) argument specifies that it should run in the background and print the container ID.

The –name argument specifies that the container should be named node_app_1.

The -p (or –publish) argument specifies to publish container port 8000 to the host port 80. By default, when you create or run a container by using docker create or docker run, it does not publish any of its ports to the outside world. The –p tag makes a port available to services outside Docker or to Docker containers that are not connected to the container's network. This technique creates a firewall rule, which maps a container port to a port on the Docker host to the outside world.

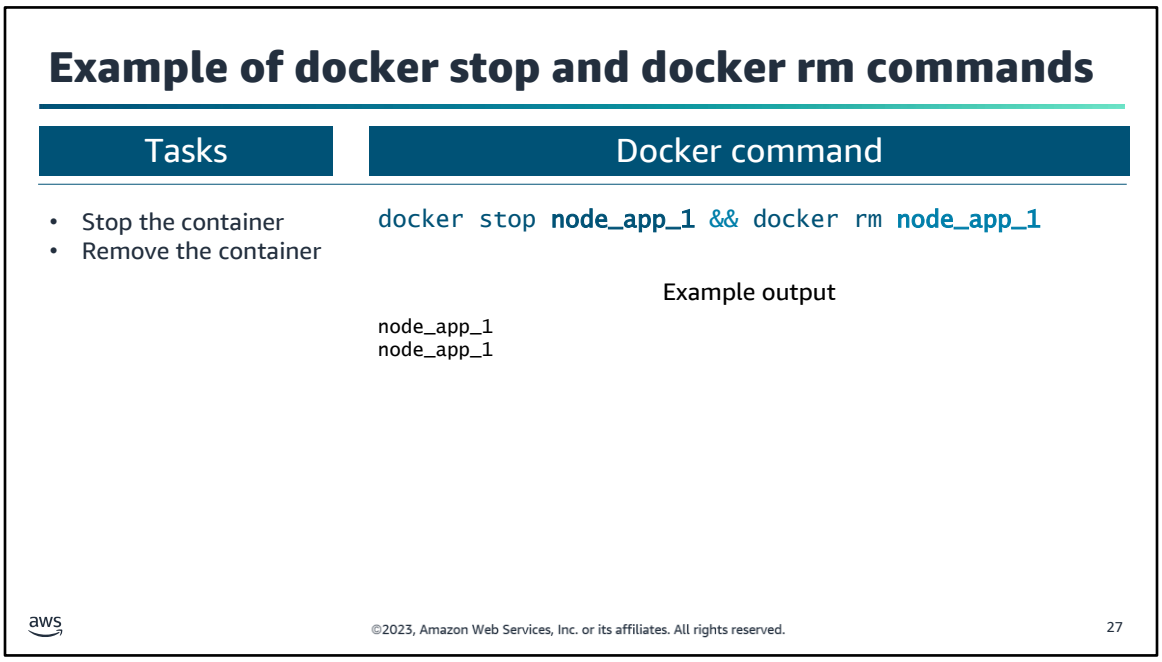
The visible output of this command is the container ID. You could follow this up with the docker ps command, as illustrated in this slide, to further verify that the container is running.



This example uses the docker exec command to open a terminal window on the running container. In this way, you can interact with the contents of the container.

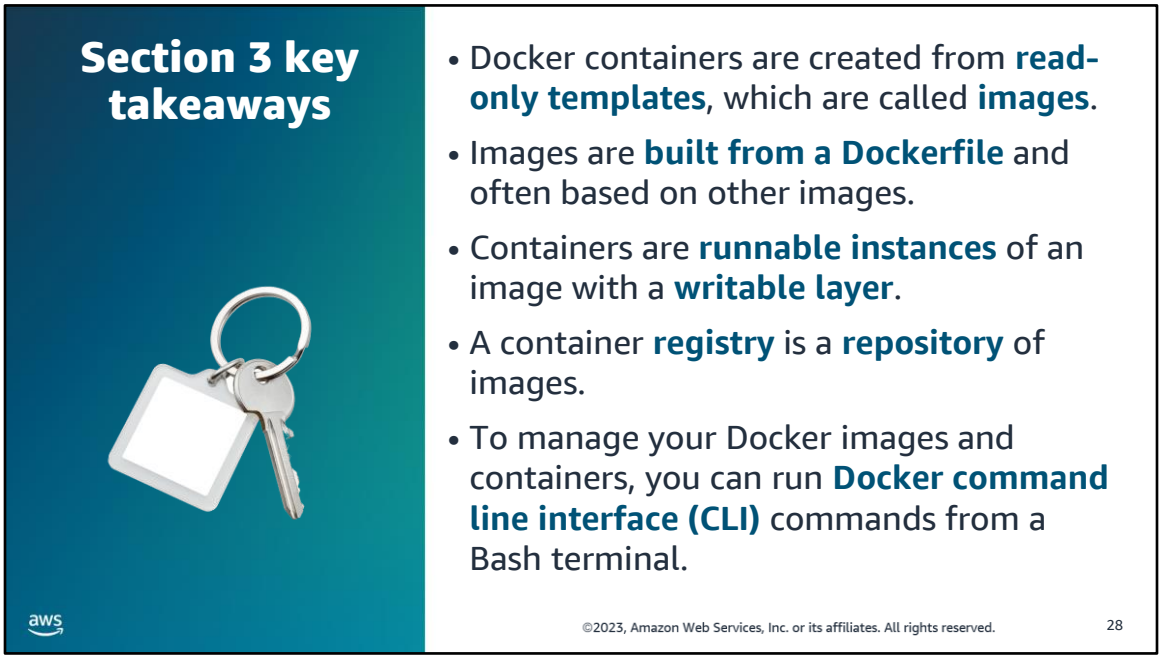
When you run docker exec with -it <container-name> sh, Docker attempts to open an interactive connection to the shell that is running on the container. The -t specifies that you want a terminal session to be invoked. The -i option specifies interactivity, so that the terminal session that you invoke will stay open. You can continue to run commands and interact with the contents of the container until you exit the terminal.

In this simple example, after connecting to the container and opening the terminal session, you use ls to list the contents, and then exit to end the session.



This example runs the docker stop and docker rm commands to stop and then remove the container that is named node_app_1.

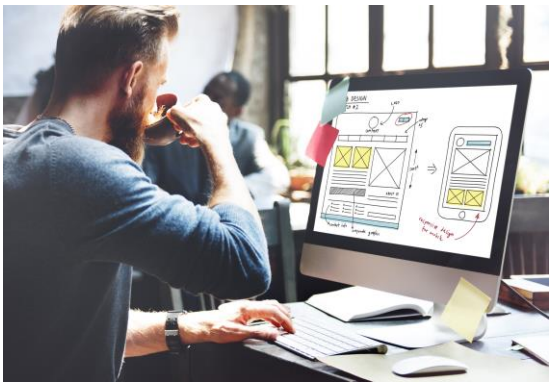
The visible output of this command is the name of the container for each part of the command (stop and remove). You could again follow this up with the docker container ls command to further verify that the container has been removed.



The following are the key takeaways from this section of the module:

- Docker containers are created from read-only templates, which are called images.
- Images are built from a Dockerfile and often based on other images.
- Containers are runnable instances of an image with a writable layer.
- A container registry is a repository of images.
- To manage your Docker images and containers, you can run Docker CLI commands from a Bash terminal.

Lab 8.1: Migrating a Web Application to Docker Containers



You will now complete Lab 8.1: Migrating a Web Application to Docker Containers.

Lab: Scenario

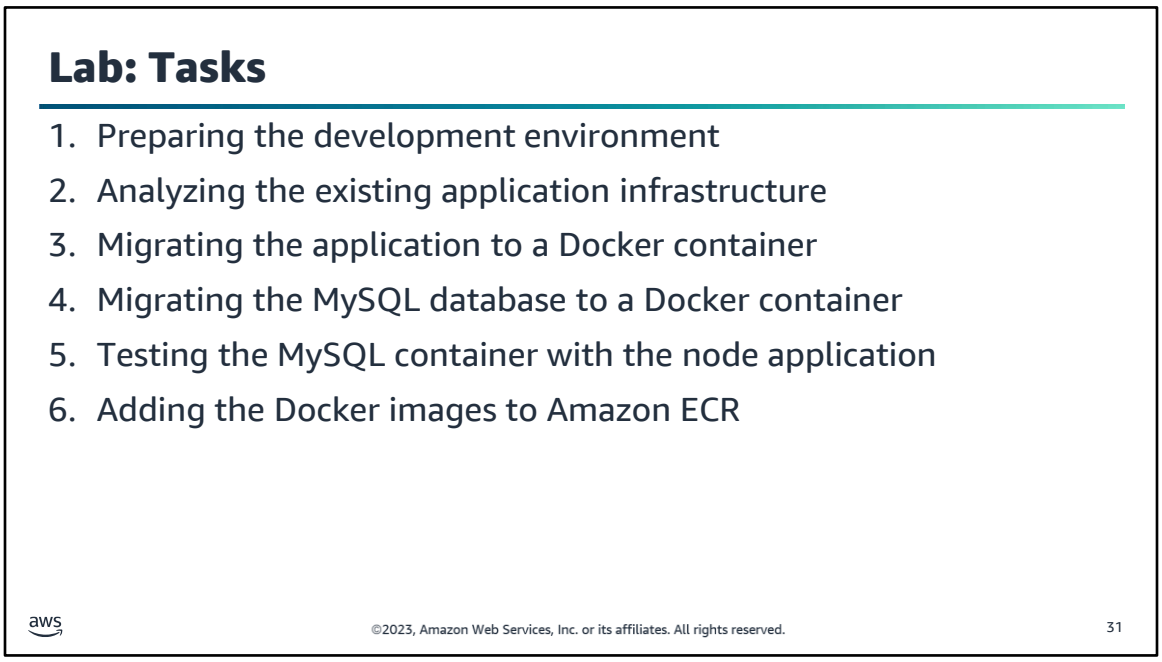
Recently, the café owners acquired one of their favorite coffee suppliers. The acquired coffee supplier runs an inventory tracking application on an AWS account.

In this lab, you again play the role of Sofía, and you will work to **migrate the application** to run on **containers**.

aws

30

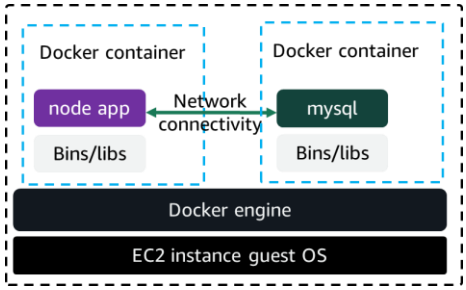
In this lab, you again play the role of Sofía, and you will work to migrate the application to run on containers.



In this lab, you will complete the following tasks:

1. Preparing the development environment
2. Analyzing the existing application infrastructure
3. Migrating the application to a Docker container
4. Migrating the MySQL database to a Docker container
5. Testing the MySQL container with the node application
6. Adding the Docker images to Amazon Elastic Container Registry (Amazon ECR)

Lab: Final product



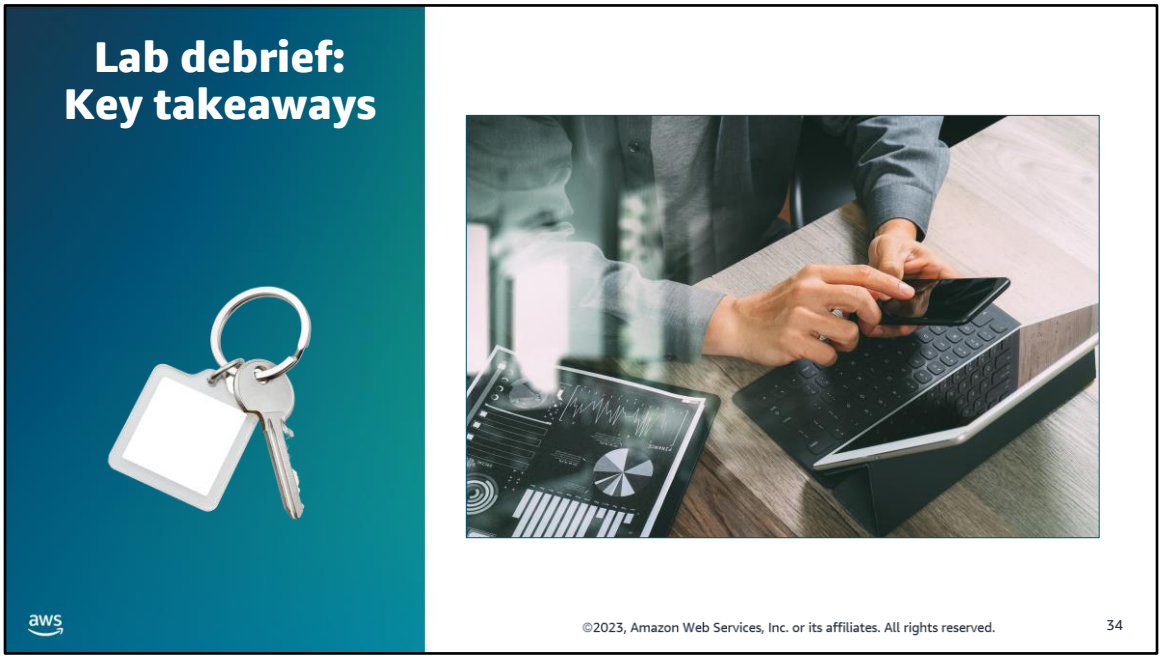
EC2 instance (being used as a container host)

 32

The diagram summarizes what you will have built after you complete the lab.

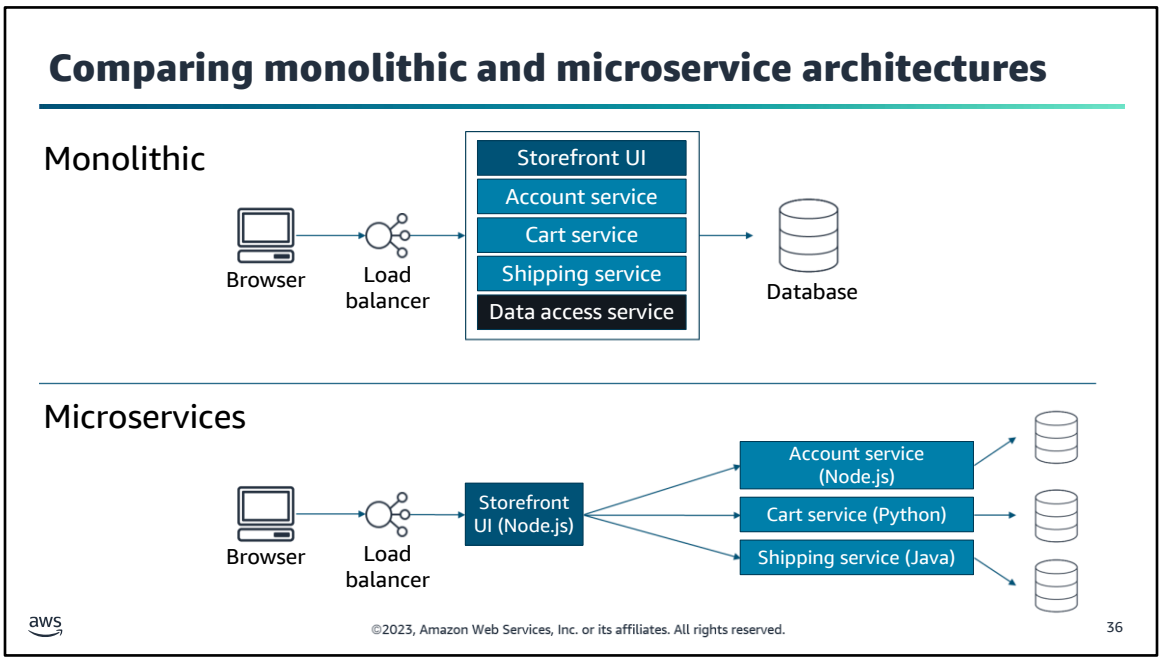
Both the application and the backend data have been migrated to run on Docker containers. The containers run on an Amazon Elastic Compute Cloud (Amazon EC2) instance.

It is now time to start the lab.



After you have completed the lab, your educator might choose to lead a conversation about the key takeaways from the lab.

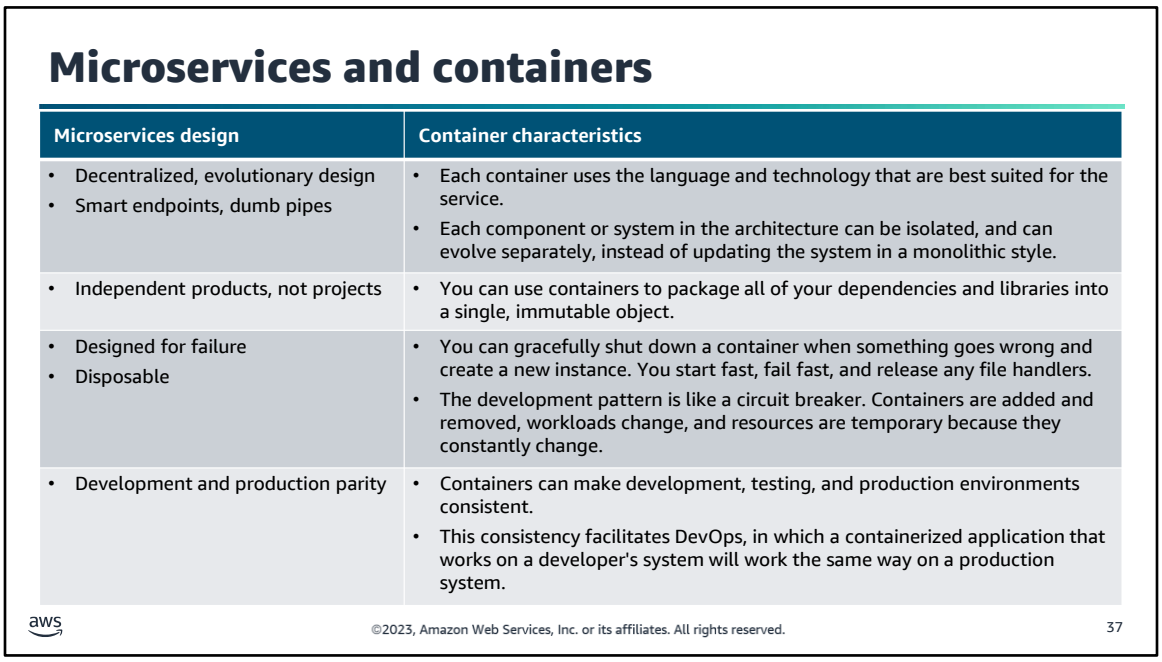
Section 4: Using containers for microservices.



One of the strongest factors that drive the growth of containers is the rise of microservice architectures. Microservices are an architectural and organizational approach to software development that is designed to speed up deployment cycles. The microservices approach fosters innovation and ownership, and improves the maintainability and scalability of software applications.

Consider this example of a *traditional monolithic architecture*. For this shopping application, all of the processes are tightly coupled. With tightly coupled processes, if one process of an application experiences a spike in demand, the entire architecture must be scaled. Adding or improving features becomes more complex as the code base grows, which limits experimentation and complicates the implementation of new ideas or technology stacks. Monolithic architectures also add risk for application availability because having many dependent and tightly coupled processes increases the impact of a single process failure.

Now, consider having the same applications run in a *microservice architecture*. Each service is built as an independent component that communicates by using lightweight API operations. Each service performs a single function that could support multiple applications. Because the services run independently, they can be updated, deployed, and scaled to meet the demand for specific functions of an application.



When you consider the characteristics of a well-designed microservice architecture, containers are well matched to support microservices. For this reason, containers are the underlying technology that powers many modern microservice architectures. Likewise, with microservice architectures, developers can take full advantage of containers.

Section 5 Introducing AWS container services.

Challenges of managing containers at scale

- State of containers
- Scheduling of starts and stops
- Resources available on each server
- Maximizing availability, resilience, and performance

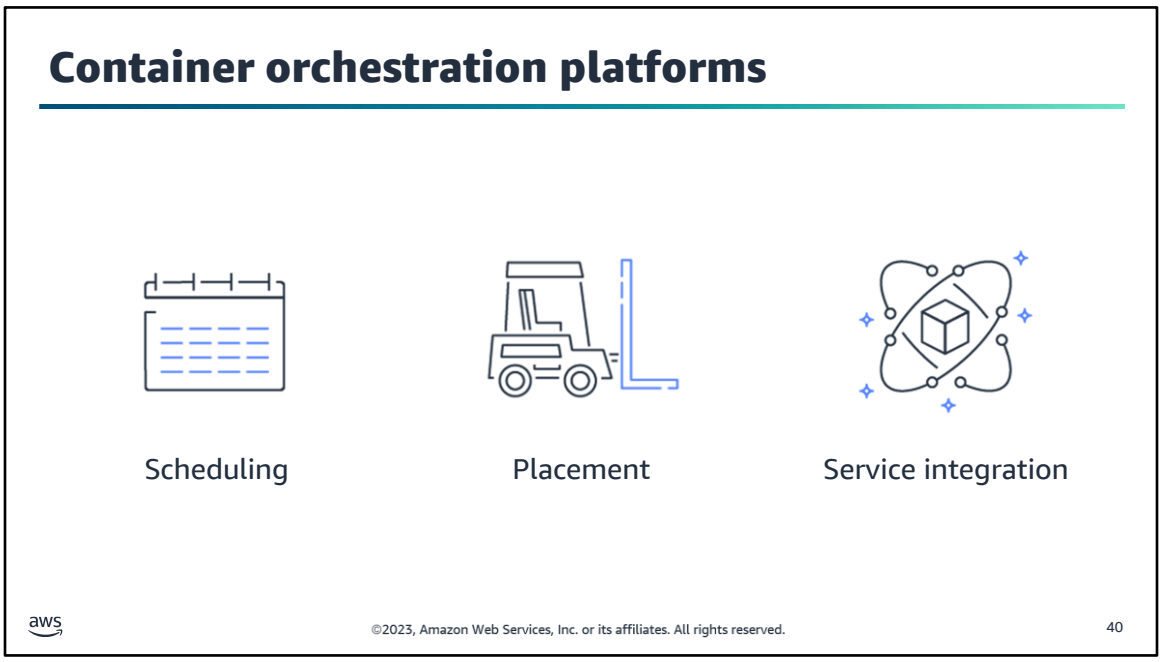
aws

39

Recall the shipping container analogy. Suppose that you have one ship and a half dozen containers. Keeping track of what's being shipped on which transport, the schedule of starts and stops, and the state of the containers is manageable. However, consider what happens when you scale up the number and type of transportation methods, along with the number of containers you are working with. Scheduling and managing which are loaded, where, and when requires a lot more logistical effort.

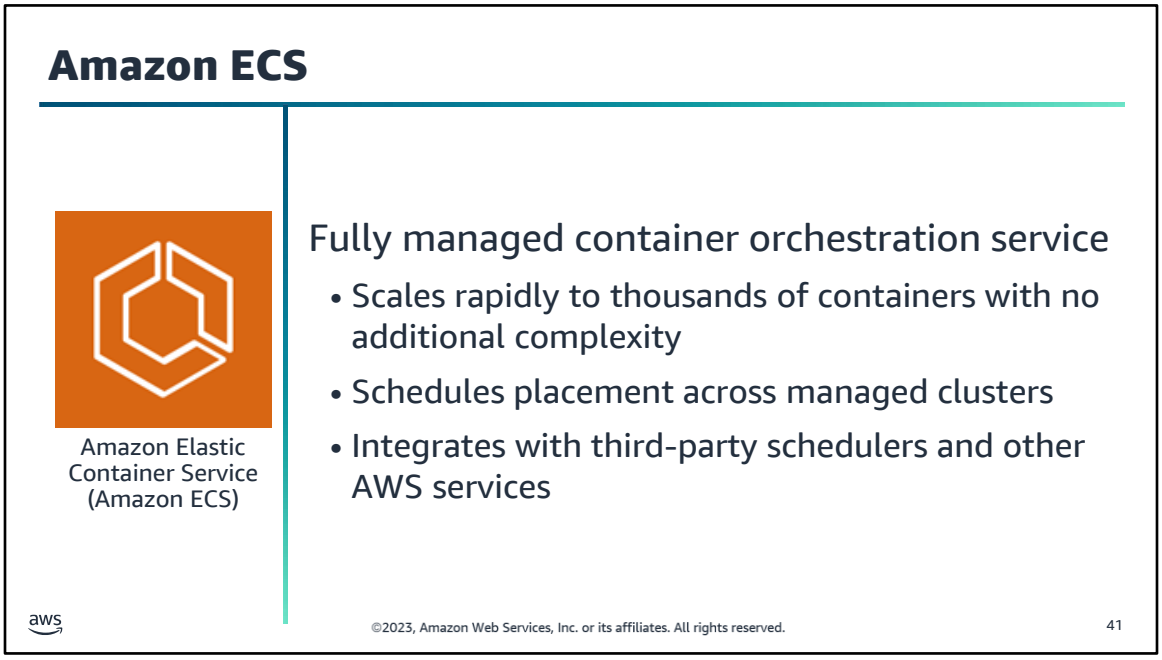
Likewise with software containers, running one or two containers on a single host is simple. But things get complex when you move into environments where you have tens of hosts with possibly hundreds of containers. Or you might have a full production environment with hundreds of hosts and maybe thousands of containers. You now must manage an enterprise-scale, clustered environment.

You must know the state of everything in your system. For example, you need to know which containers aren't working, and which are starting and stopping. You need to know where you have enough room and memory to add new containers. Finally, you need a way to place your containers intelligently on instances to maximize availability, resilience, and performance.



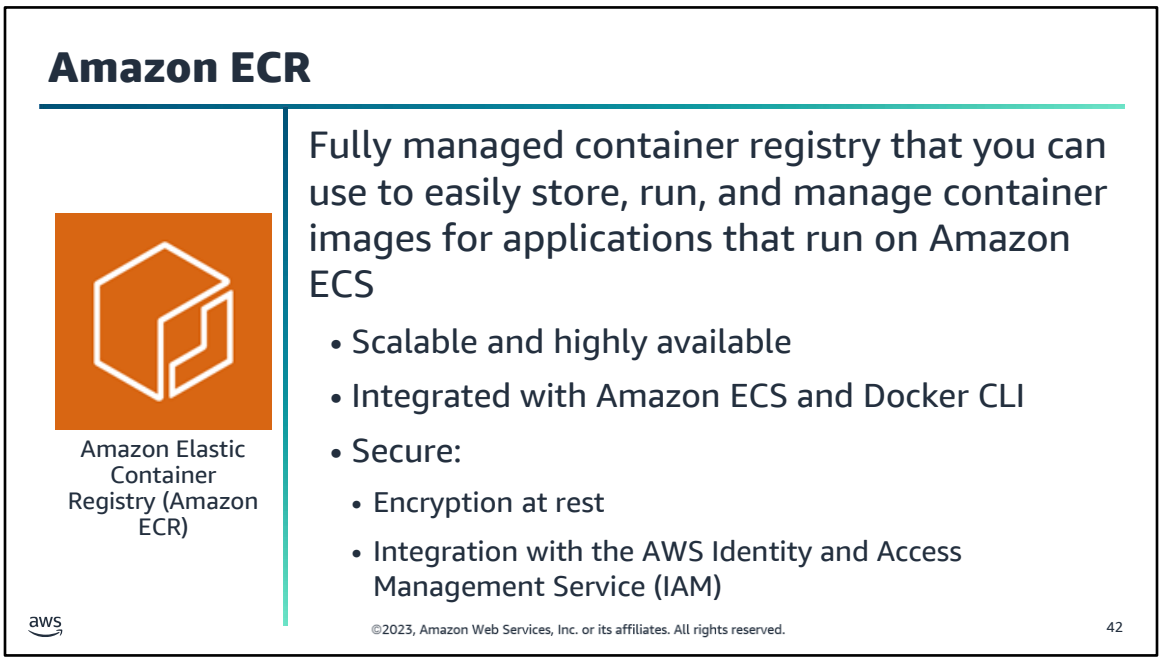
This situation is where *container orchestration* platforms become useful. They handle the scheduling and placement of containers based on the underlying hardware infrastructure and needs of the application. Container orchestration platforms integrate with other services, such as services for networking, persistent storage, security, monitoring, and logging.

Many options exist, including native tools like Docker Swarm, open-source platforms like Kubernetes, and Amazon Elastic Container Service (Amazon ECS). The orchestration platform is arguably the most important choice that you will make when you architect a container-based workload.



Amazon ECS is a highly scalable, highly performing container orchestration service that supports Docker containers. You can use the service to run and scale containerized applications on AWS. You can use Amazon ECS to schedule the placement of containers across a managed cluster of EC2 instances. Amazon ECS provides its own schedulers, but it can also integrate with third-party schedulers to meet business or application-specific requirements. Amazon ECS is also tightly integrated with other AWS services, such as AWS Identity and Access Management (IAM), Amazon CloudWatch, and Amazon Route 53.

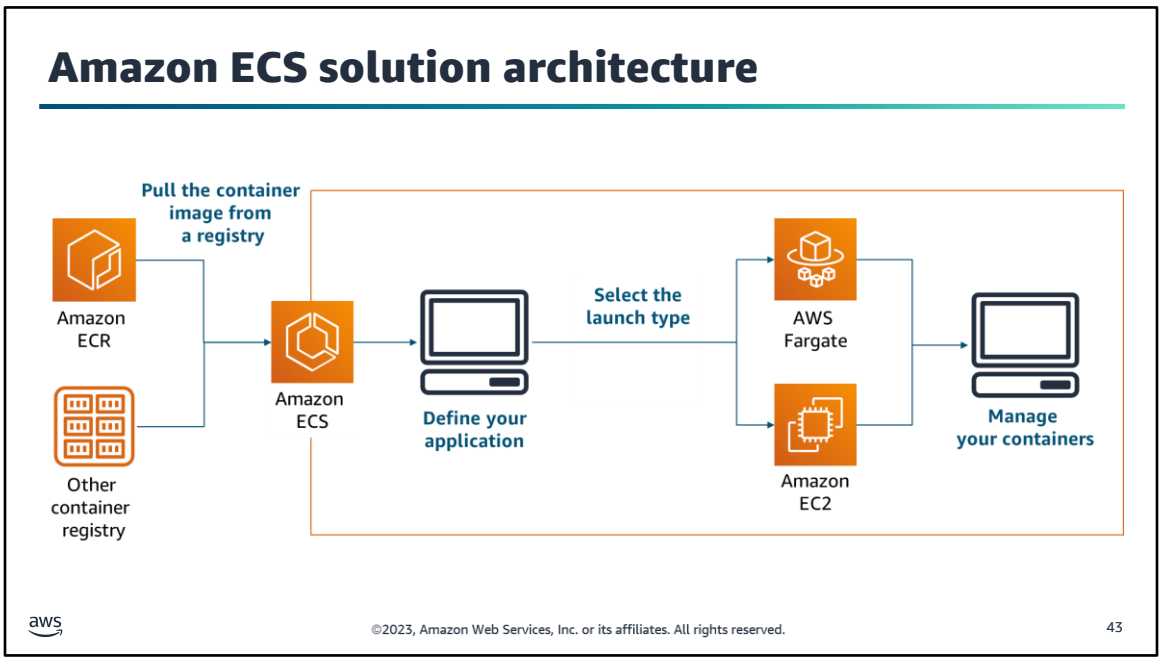
For more information about Amazon ECS, see the product page at https://aws.amazon.com/ecs/.



Amazon ECR is a fully managed, cloud-based Docker image registry that makes it easy for you to store, manage, and deploy Docker container images. Amazon ECR integrates with Amazon ECS and the Docker CLI, which simplifies your development and production workflows. You can push your container images to Amazon ECR by using the Docker CLI from your development machine. Then, Amazon ECS can pull them directly for production deployments.

Amazon ECR reduces the need to operate your own container repositories or be concerned about scaling the underlying infrastructure. Amazon ECR hosts your images in a highly available and scalable architecture, which enables you to reliably deploy containers for your applications. Amazon ECR is also secure. Amazon ECR transfers your container images over HTTPS, and it automatically encrypts your images at rest. You can configure policies to manage permissions for each repository and restrict access to IAM users, roles, or other AWS accounts.

For more information about Amazon ECR, see the product page at https://aws.amazon.com/ecr/.



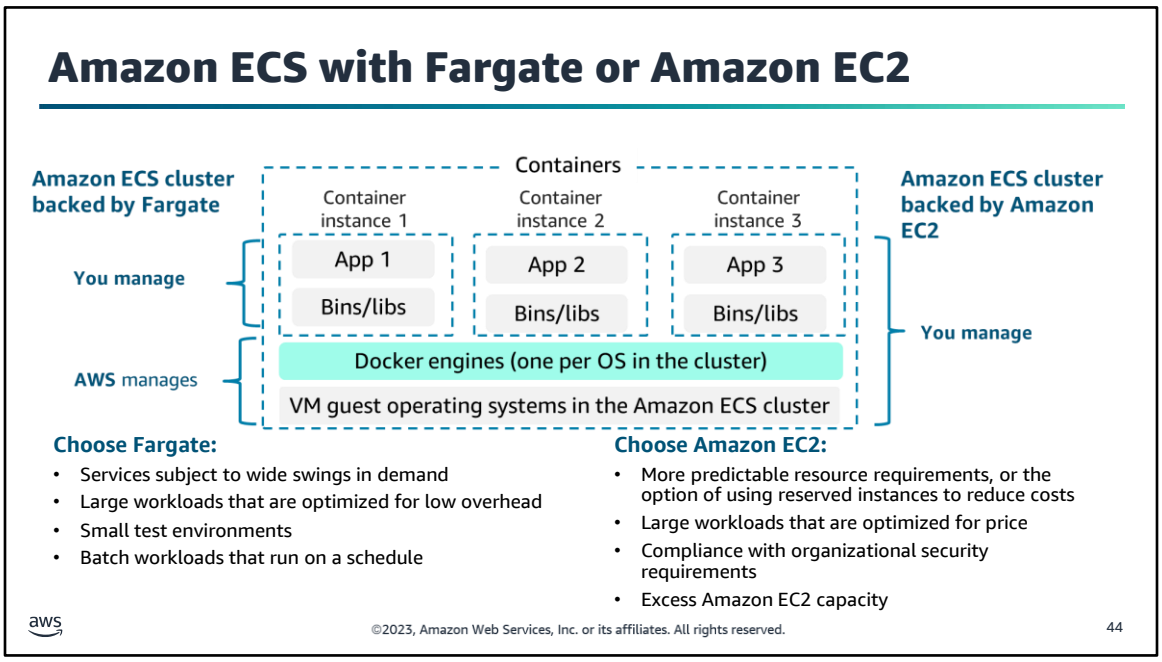
The following procedure is a high-level overview of Amazon ECS.

First, container images are pulled from a registry. This registry can be Amazon ECR—which is one of many AWS services that integrate with Amazon ECS—or a third-party or private registry.

Next, you define your application. Customize the container images with the necessary code and resources, and then create the appropriate configuration files to group. Then, define your containers as short-running tasks or long-running services within Amazon ECS.

When you are ready to bring your services online, you select one of two launch types: AWS Fargate or Amazon EC2. You can mix and match the two launch types as needed within your application. The next slide highlights distinctions between these two launch types.

Finally, you can use Amazon ECS to manage your containers. Amazon ECS scales your application and manages your containers for availability.



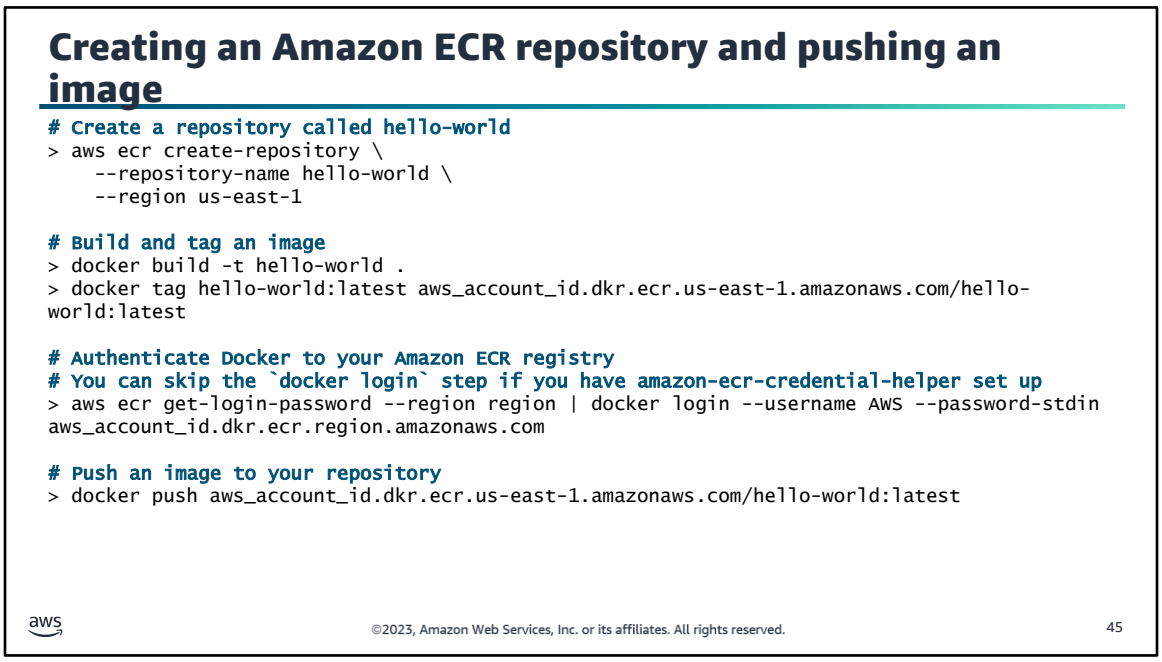
The Fargate launch type provides a near-serverless experience, where AWS completely manages the infrastructure that supports your containers. AWS manages the placement of your tasks on instances, and makes sure that each task has the appropriate amount of CPU and memory. With Fargate, you can focus on the tasks and the application architecture, instead of worrying about the infrastructure.

The Amazon EC2 launch type is useful when you want more control over the infrastructure that supports your tasks. When you use the Amazon EC2 launch type, you create and manage clusters of EC2 instances to support your containers. You also define the placement of containers across your cluster based on your resource needs, isolation policies, and availability requirements. You have more granular control over your environment without operating your own cluster management and configuration management systems, or worrying about scaling your management infrastructure.

For example, you can use EC2 to launch services that have more predictable resource requirements. You can use Fargate to launch other services that are subject to wide swings in demand. Regardless of the launch type that you use, Amazon ECS manages your containers for availability, and it scales your application as necessary to meet demand.

For more information about AWS Fargate, see the product page at https://aws.amazon.com/fargate/.

For more information about the Fargate and EC2 launch types, see https://docs.aws.amazon.com/AmazonECS/latest/developerguide/launch_types.html .

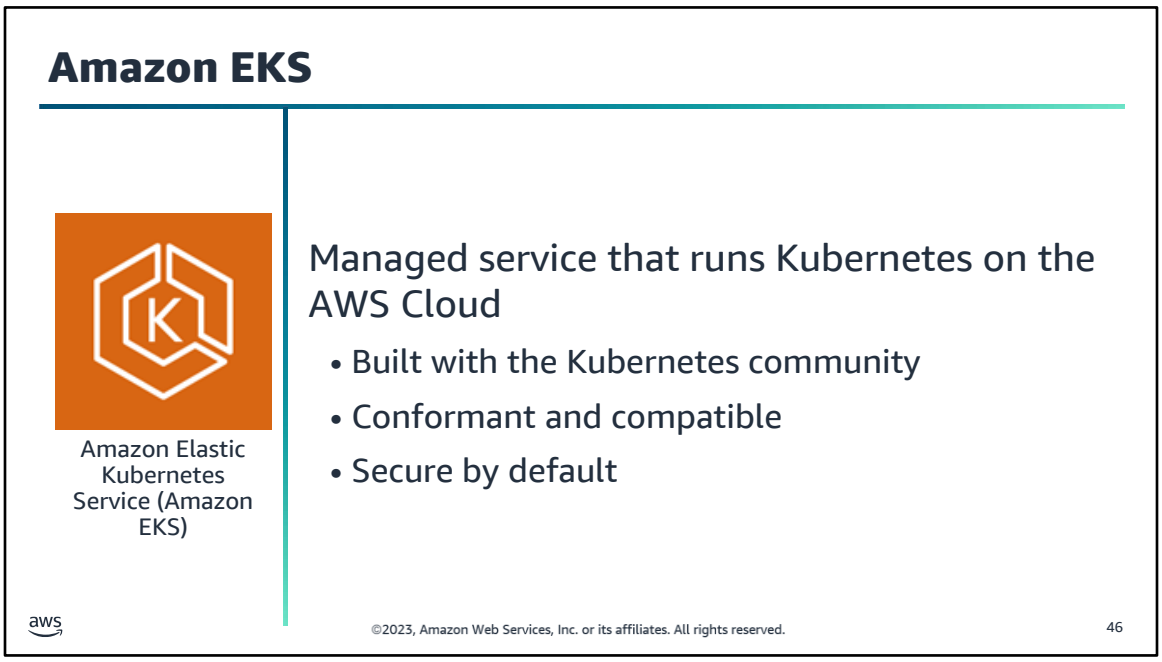


You can programmatically access Amazon ECR from the AWS Command Line Interface (AWS CLI) and by using the APIs. You can use the AWS CLI and APIs to create, monitor, and delete repositories and set repository permissions. You can perform these same actions in the Amazon ECR console, which you can access from the Amazon ECS console. Amazon ECR integrates with the Docker CLI, which allows you to push, pull, and tag images on your development machine.

This example shows how to create a repository that is called *hello-world* with Amazon ECR. The example uses Docker CLI commands to build and tag an image, and then push it into the repository. To push an image into Amazon ECR, you must first authenticate the Docker client to your Amazon ECR registry.

For more information, see the following resources:

- Using Amazon ECR with the AWS CLI: https://docs.aws.amazon.com/AmazonECR/latest/userguide/getting-started-cli.html.
- Authenticating Amazon ECR Repositories for Docker CLI with Credential Helper: https://aws.amazon.com/blogs/compute/authenticating-amazon-ecr-repositories-for-docker-cli-with-credential-helper/.

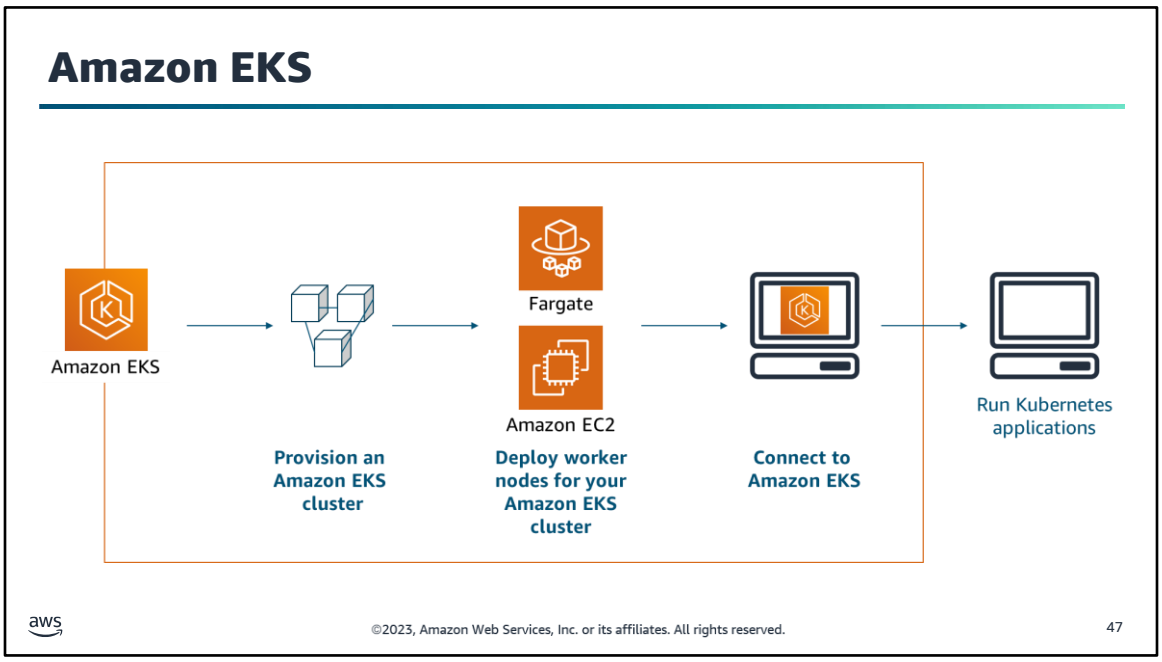


Kubernetes is an open-source container management platform that you can use to deploy and manage containerized applications at scale. Kubernetes manages clusters of EC2 instances, and it runs containers on those instances with processes for deployment, maintenance, and scaling. By using Kubernetes, you can run any type of containerized application by using the same toolset on premises and in the cloud.

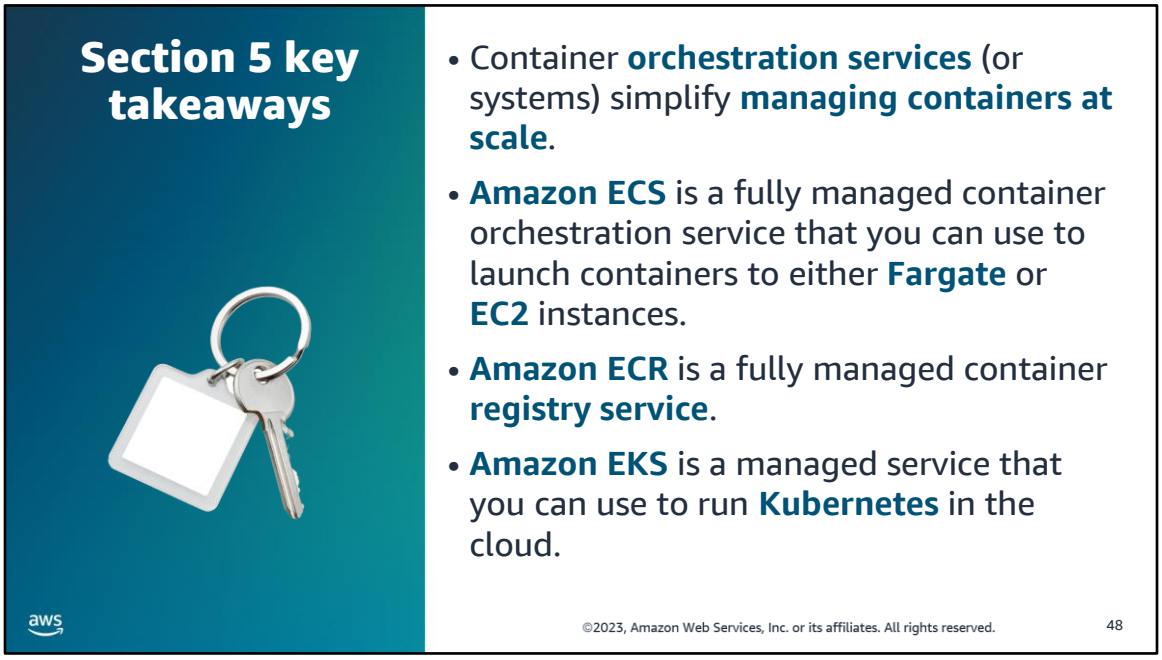
You can use AWS services to run Kubernetes in the cloud. These services include scalable and highly available VM infrastructure, community created service integrations, and Amazon Elastic Kubernetes Service (Amazon EKS). Amazon EKS is a managed service that runs the Kubernetes management infrastructure across multiple Availability Zones to reduce the chance of a single point of failure.

Amazon EKS is certified Kubernetes conformant, so you can use existing tools and plugins from partners and the Kubernetes community. Applications that run on any standard Kubernetes environment are fully compatible, and they can be migrated to Amazon EKS. Amazon EKS automatically sets up secure and encrypted channels to your worker nodes, which makes your infrastructure that runs on Amazon EKS secure by default. AWS actively works with the Kubernetes community by contributing to the Kubernetes code base, which helps Amazon EKS users employ AWS services and features.

For more information about Amazon EKS, see the product page at https://aws.amazon.com/eks/.



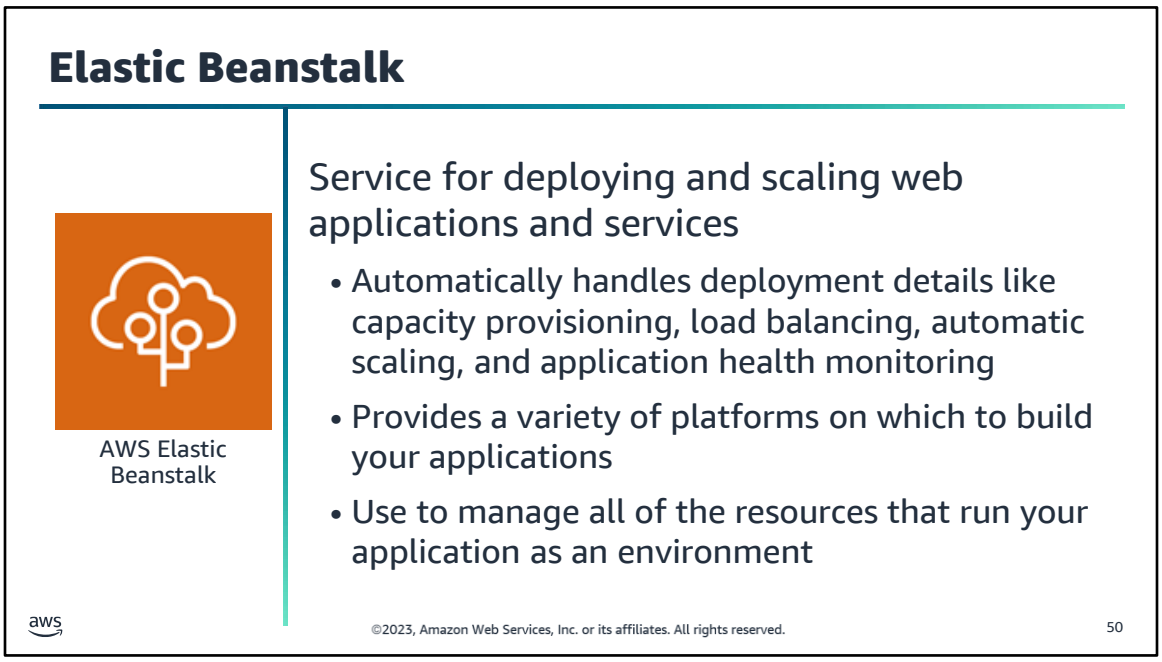
When you select Amazon EKS as your container management service, you provision an Amazon EKS cluster and deploy Amazon EC2 or Fargate worker nodes (that is, worker machines) for your Amazon EKS cluster. You then connect to Amazon EKS and run your Kubernetes applications.



The following are the key takeaways from this section of the module:

- Container orchestration services (or systems) simplify managing containers at scale.
- Amazon ECS is a fully managed container orchestration service that you can use to launch containers to either Fargate or EC2 instances.
- Amazon ECR is a fully managed container registry service.
- Amazon EKS is a managed service that you can use to run Kubernetes in the cloud.

Section 6 Deploying applications with Elastic Beanstalk.



Elastic Beanstalk is a service for deploying and scaling web applications and services. It automatically handles deployment details like capacity provisioning, load balancing, auto scaling, and application health monitoring.

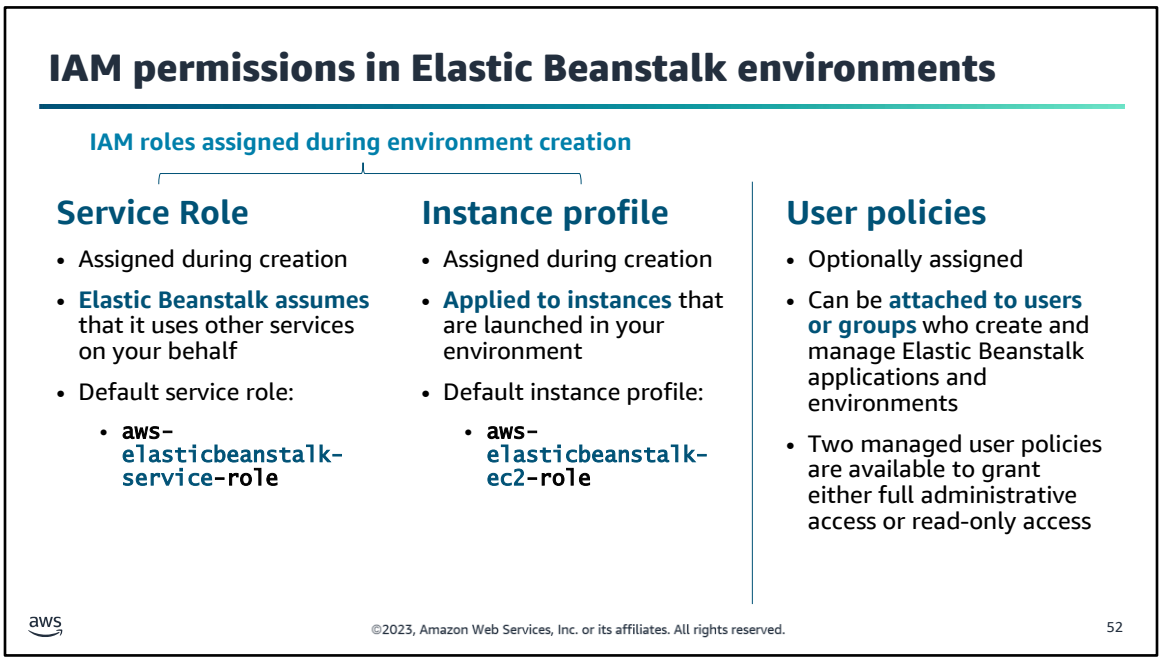
Elastic Beanstalk components

Command	Description
Application	Logical collection of Elastic Beanstalk components. Conceptually similar to a folder.
Application version	Specific, labeled iteration of deployable code for a web application.
Environment	Collection of AWS resources that run an application version.
Environment tier	Designation of the type of application that the environment runs. Determines what resources Elastic Beanstalk provisions to support it.
Environment configuration	Collection of parameters and settings that define how an environment and its associated resources behave.
Saved configuration	Template that you can use as a starting point for creating unique environment configurations.
Platform	Combination of an OS, programming language runtime, web server, application server, and Elastic Beanstalk components. You design and target your web application to a platform.
Elastic Beanstalk CLI	CLI for Elastic Beanstalk. Provides interactive commands that simplify creating, updating, and monitoring environments from a local repository.

aws

51

AWS Elastic Beanstalk enables you to manage all of the resources that run your application as environments. This slide lists key Elastic Beanstalk components.



The Elastic Beanstalk permissions model requires you to assign two roles when creating an environment: the service role and the instance profile.

Service role:
Elastic Beanstalk assumes the service role to use other AWS services on your behalf.

Suppose that you create an environment with the eb create command in the Elastic Beanstalk CLI. If you don't specify a service role through the --service-role option, Elastic Beanstalk creates the default service role, aws-elasticbeanstalk-service-role. If the default service role already exists, Elastic Beanstalk uses it for the new environment.

Instance profile:
An instance profile is a container for an IAM role that can pass role information to an EC2 instance when the instance starts.

The instance profile is applied to the instances in your environment. The instances retrieve application versions from Amazon Simple Storage Service (Amazon S3), upload logs to Amazon S3, and perform other tasks. The tasks vary depending on the environment type and platform. In multicontainer Docker environments, the instance profile coordinates container deployments with Amazon ECS.

When you launch an environment by using the Elastic Beanstalk console or the CLI, Elastic Beanstalk creates a default instance profile, called aws-elasticbeanstalk-ec2-role. Managed policies are assigned to the role with default permissions.

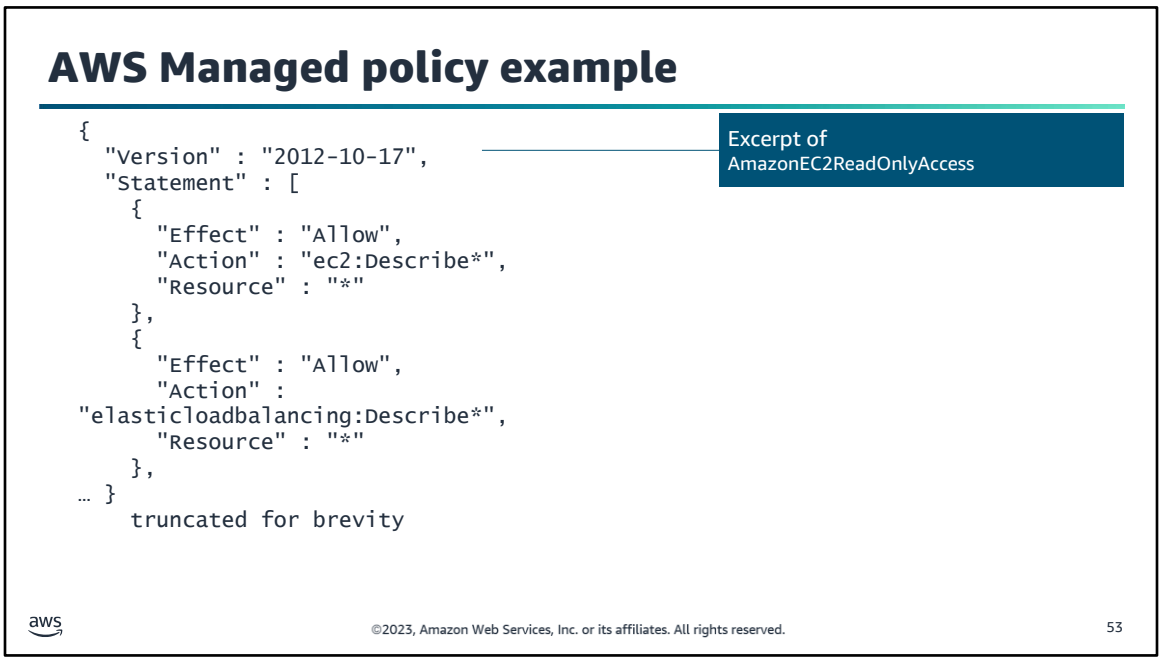
Elastic Beanstalk provides three managed policies that are assigned to your environment by default. The policies consist of one for the web server tier, one for the worker tier, and one with additional permissions necessary for multicontainer Docker environments:

- AWSElasticBeanstalkWebTier grants permissions to instances in your environment to upload logs to Amazon S3 and send debugging information to AWS X-Ray.
- AWSElasticBeanstalkWorkerTier grants permissions for log uploads, debugging, metric publication, and worker instance tasks, including queue management, leader election, and periodic tasks.
- AWSElasticBeanstalkMulticontainerDocker grants permissions for Amazon ECS to coordinate cluster tasks.

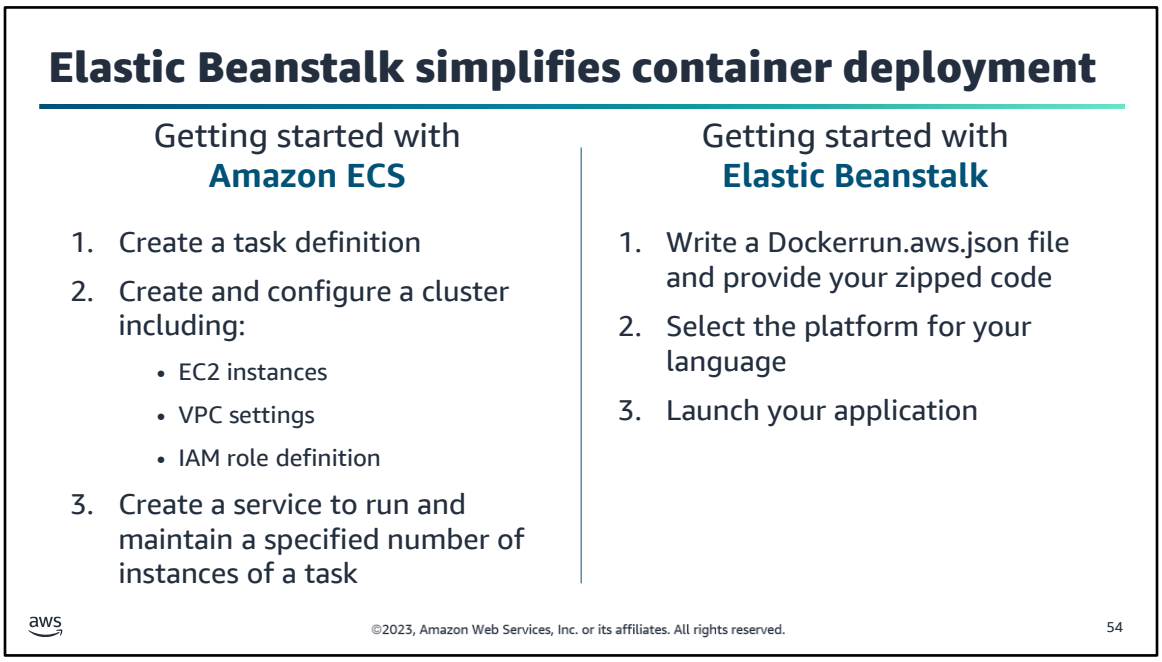
User policies:

User policies enable users to create and manage environments. Elastic Beanstalk provides managed policies for full access and read-only access:

- AdministratorAccess-AWSElasticBeanstalk
- AWSElasticBeanstalkReadOnly



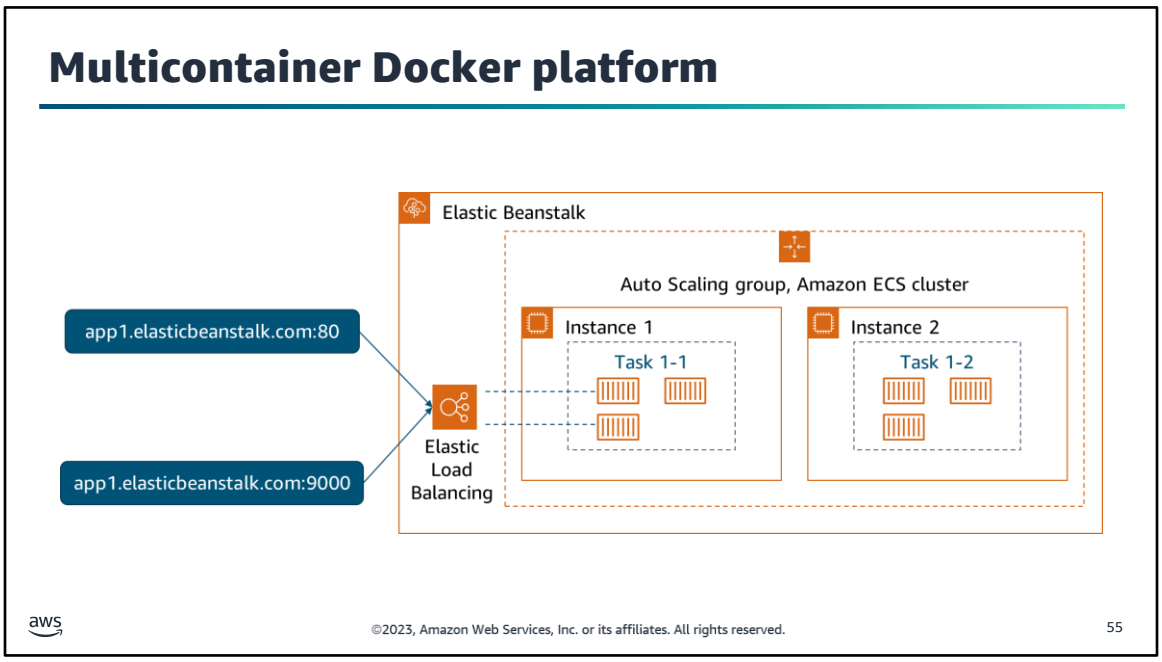
This slide shows an excerpt of the AmazonEC2ReadOnlyAccess policy.



Elastic Beanstalk uses Amazon ECS to coordinate container deployments to multicontainer Docker environments. This coordination simplifies the task of getting up and running with containers. Elastic Beanstalk takes care of Amazon ECS tasks, including cluster creation, task definition, and running tasks.

You can also continue to customize your Elastic Beanstalk environment as you experiment and learn more about working with containers. When you create an Elastic Beanstalk environment, Elastic Beanstalk provisions and configures all of the AWS resources necessary to run and support your application. In addition to configuring your environment's metadata and update behavior, you can customize these resources by providing values for configuration options.

Elastic Beanstalk uses AWS CloudFormation to launch the resources in your environment and propagate configuration changes. The resources are defined in a template that you can view in the CloudFormation console.



You can use Elastic Beanstalk to create an environment where your EC2 instances run multiple Docker containers side by side.

This diagram shows an example Elastic Beanstalk environment that is configured with three Docker containers running on each EC2 instance in an Auto Scaling group.

The multicontainer Docker platform for Elastic Beanstalk requires images to be prebuilt and stored in a public or private online image repository.

When you create an environment by using the multicontainer Docker platform, Elastic Beanstalk automatically creates and configures several Amazon ECS resources. At the same time, it is building the environment to create the necessary containers on each EC2 instance.

Here are the components that Elastic Beanstalk creates:

- **Amazon ECS cluster:** Container instances in Amazon ECS are organized into clusters. When they are used with Elastic Beanstalk, one cluster is always created for each multicontainer Docker environment.
- **Amazon ECS task definition:** Elastic Beanstalk uses the Dockerrun.aws.json file in your project to generate the Amazon ECS task definition that configures container instances in the environment.
- **Amazon ECS task:** Elastic Beanstalk communicates with Amazon ECS to run a task on every instance in the environment to coordinate container deployment. In a scalable environment, Elastic Beanstalk initiates a new task whenever an instance is added to the cluster. In rare cases, you might need to increase the amount of space that is reserved for containers and images.
- **Amazon ECS container agent:** The agent runs in a Docker container on the instances in your environment. The agent polls the Amazon ECS service and waits

for a task to run.

- **Amazon ECS data volumes:** Elastic Beanstalk inserts volume definitions (in addition to the volumes that you define in Dockerrun.aws.json) into the task definition to facilitate log collection.
- Elastic Beanstalk creates log volumes on the container instance, one for each container, at /var/log/containers/containername. These volumes are named awseb-logs-containername and are provided for containers to mount.

Service role policy example

```
{
  "AWSEBDockerrunVersion": 2,
  "volumes": [
    {
      "name": "php-app",
      "host": {
        "sourcePath": "/var/app/current/php-
  app"
      }
    },
    {
      "name": "nginx-proxy-conf",
      "host": {
        "sourcePath":
  "/var/app/current/proxy/conf.d"
      }
    }
  ]
…
```

```
"containerDefinitions": [
    {
      "name": "php-app",
      "image": "php:fpm",
      "environment": [
        {
          "name": "Container",
          "value": "PHP"
        }
      ],
      "essential": true,
      "memory": 128,
      "mountPoints": [
        {
          "sourceVolume": "php-app",
          "containerPath": "/var/www/html",
          "readOnly": true
        }
      ]
    }
…
```

aws

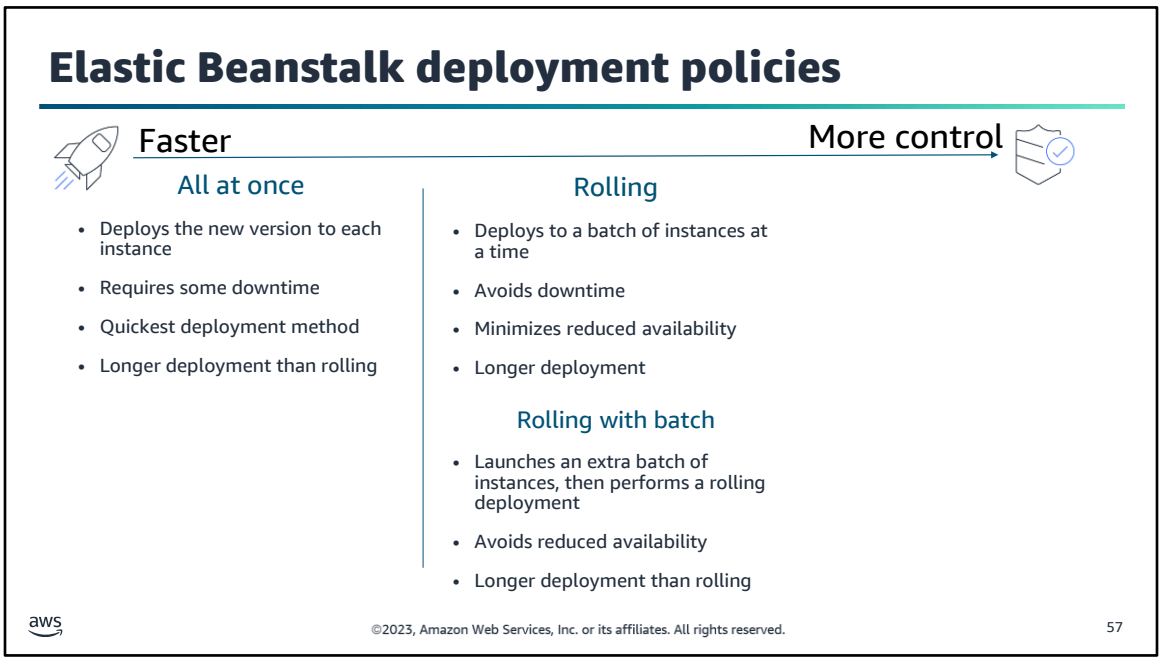
56

Container instances are EC2 instances that run multicontainer Docker in an Elastic Beanstalk environment. Container instances require a configuration file that is named Dockerrun.aws.json. This file is specific to Elastic Beanstalk. The file can be used alone or combined with source code and content in a source bundle to create an environment on a Docker platform.

The Dockerrun.aws.json file includes three sections:

- AWSEBDockerrunVersion: Specifies the version number as the value 2 for multicontainer Docker environments.
- volumes: Creates volumes from folders in the EC2 container instance or from your source bundle (deployed to /var/app/current). Mount these volumes to paths within your Docker containers by using mountPoints in the container definition.
- containerDefinitions: An array of container definitions. The container definition and volumes sections of Dockerrun.aws.json use the same formatting as the corresponding sections of an Amazon ECS task definition file.

This example uses an instance with two containers. The snippet on the left illustrates the syntax for the AWSEBDockerrunVersion and volumes sections. The snippet on the right shows the continuation of the file with the containerDefinitions section. Only the first definition is shown in the snippet. For the full example, see the Elastic Beanstalk Developer Guide at https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/create_deploy_docker_v2config.html#create_deploy_docker_v2config_dockerrun.

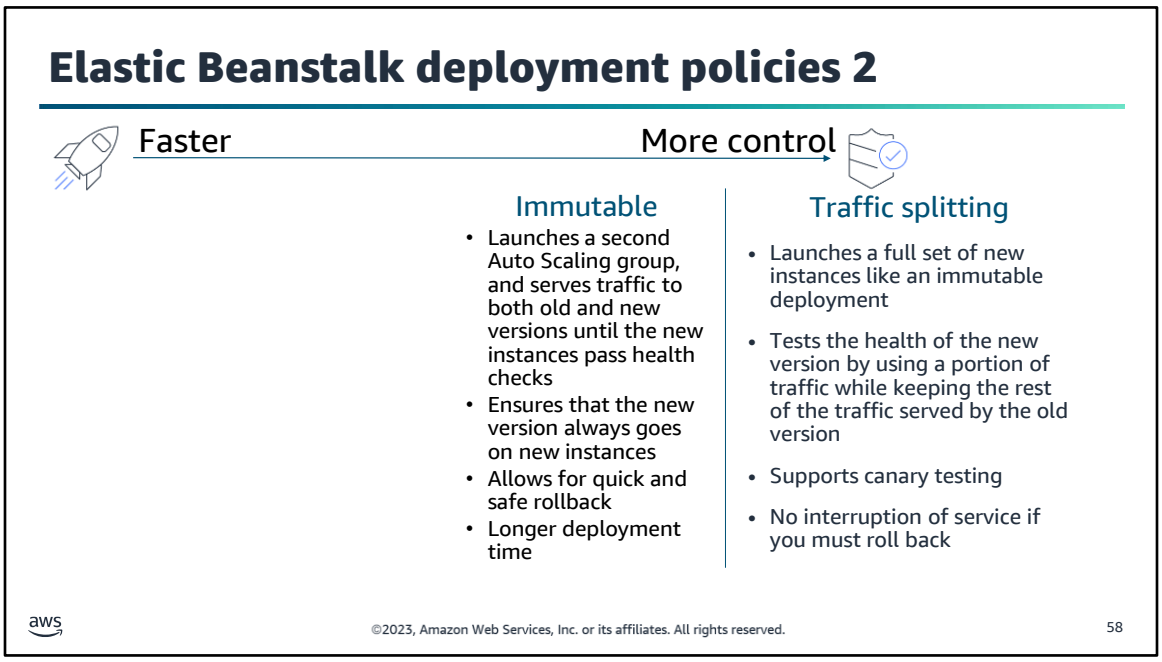


Elastic Beanstalk provides several options for processing deployments, including deployment policies (all at once, rolling, rolling with additional batch, immutable, and traffic splitting). It also includes options that you can use to configure batch size and health check behavior during deployments.

Choose the method that makes the most sense for your use case. Consider how fast you can make updates compared to how much tolerance your users have for issues with a new version and downtime during updates.

All at once deployments are the fastest but require at least some downtime.

If you created the environment with the CLI and it's a scalable environment (you didn't specify the --single option), it uses rolling deployments.

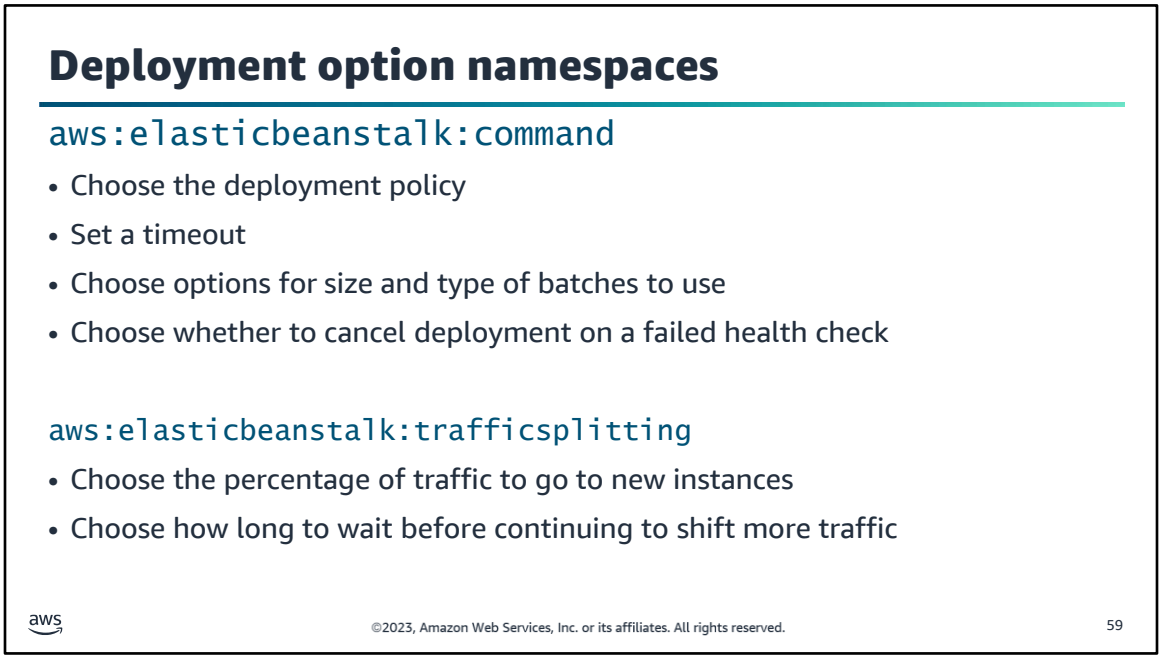


Immutable deployments are the safest but require a longer deployment time.

With traffic splitting, you can test a small amount of traffic to the new version while still sending most traffic to the existing version. Then, you can automatically shift all traffic to the new version after a period if health checks are successful.

By default, your environment uses all at once deployments.

For more information about these options, see the Elastic Beanstalk Developer Guide at https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.deploy-existing-version.html.



Elastic Beanstalk defines a large number of configuration options that you can use to configure your environment's behavior and the resources that it contains. Configuration options are organized into namespaces.

You can use the configuration options in the aws:elasticbeanstalk:command namespace to configure your deployments.

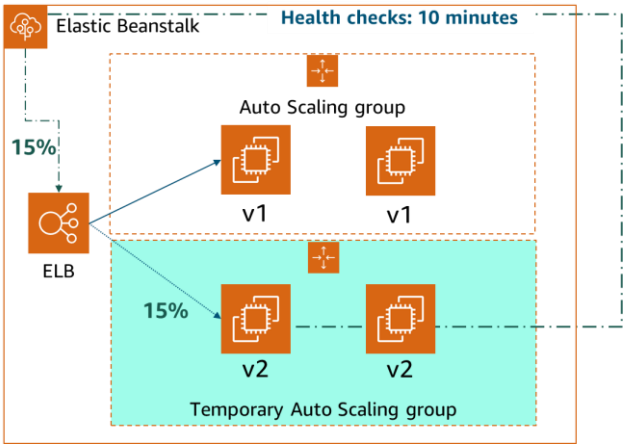
If you choose the traffic-splitting policy, additional options for this policy are available in the aws:elasticbeanstalk:trafficsplitting namespace.

These deployment options can be set by using configuration files (.ebextensions).

Example of traffic splitting (canary testing)

Example deployment configurations

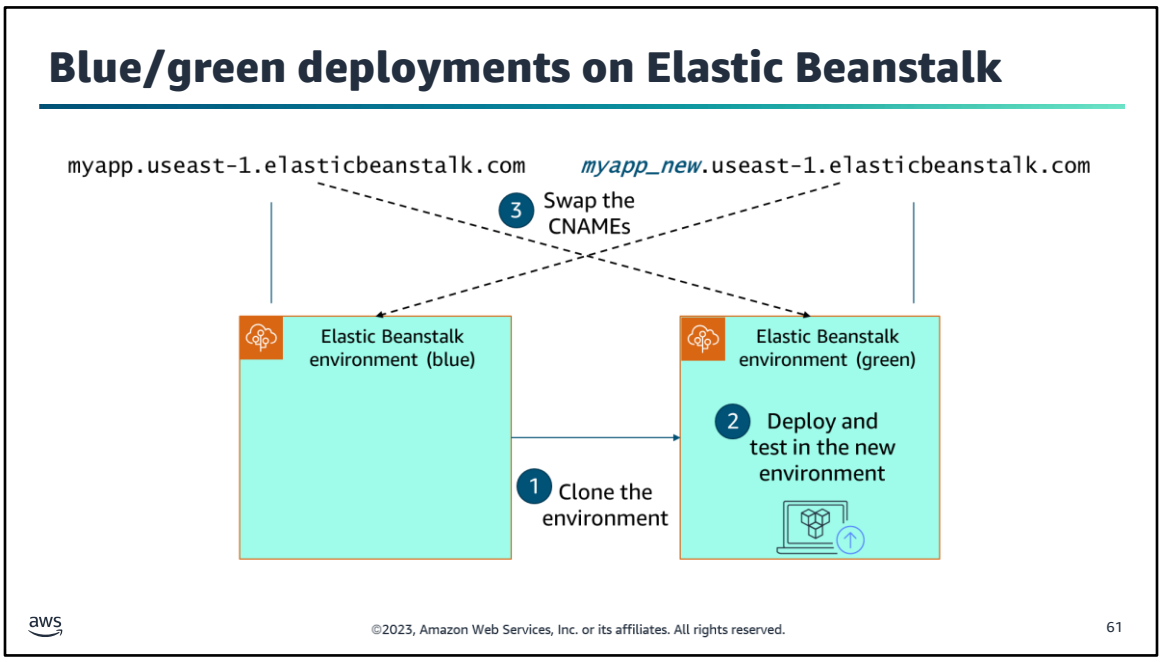
```
option_settings:
  aws:elasticbeanstalk:command:
    DeploymentPolicy: TrafficSplitting

aws:elasticbeanstalk:trafficsplitting:
    NewVersionPercent: "15"
    EvaluationTime: "10"
```



You can use traffic-splitting deployments to perform canary testing. You direct some incoming client traffic to your new application version to verify the application's health. As soon as the application's health is verified, you can commit to the new version and direct all traffic to it.

During a traffic-splitting deployment, Elastic Beanstalk creates a new set of instances in a separate temporary Auto Scaling group. Elastic Beanstalk instructs the load balancer to direct a certain percentage of your environment's incoming traffic (NewVersionPercent) to the new instances. The decision is based on the configuration values in the aws:elasticbeanstalk:trafficsplitting namespace of your configuration file.

Then, for the configured amount of time (EvaluationTime), Elastic Beanstalk tracks the health of the new set of instances. If all is well, Elastic Beanstalk shifts remaining traffic to the new instances and attaches them to the environment's original Auto Scaling group. They replace the old instances. Then, Elastic Beanstalk cleans up—it terminates the old instances and removes the temporary Auto Scaling group.

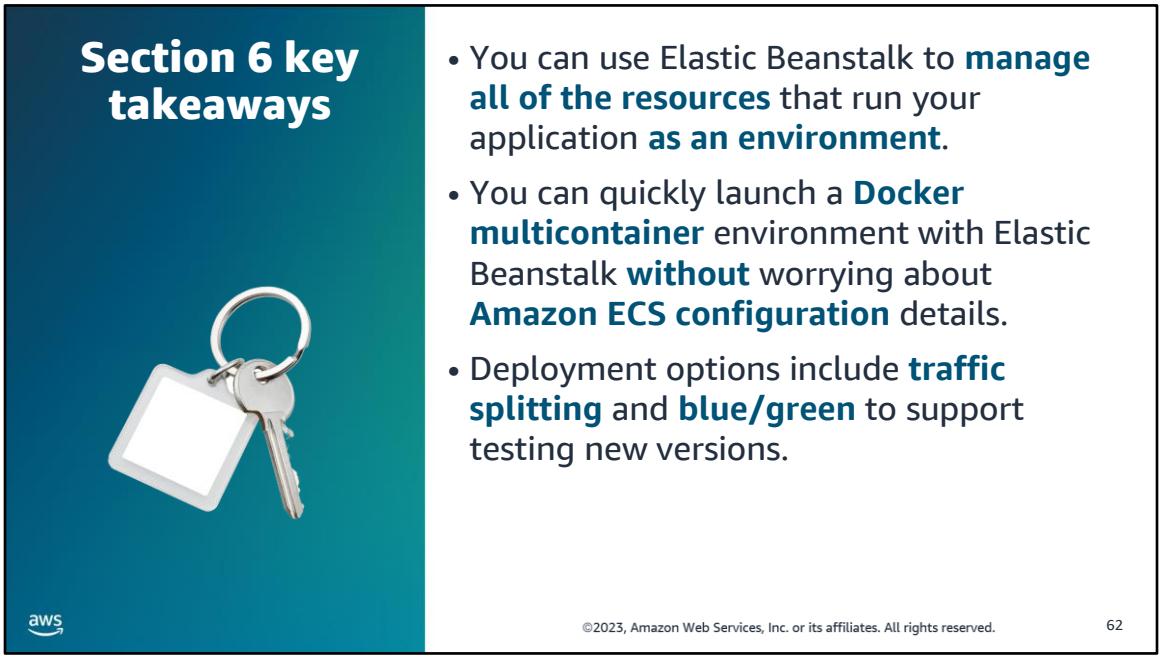


Blue/green deployments are similar to immutable deployments because you create an entirely new environment and then move your application to that environment.

With a blue/green deployment, you deploy the new version to a separate environment. You then swap CNAMEs of the two environments to redirect traffic to the new version instantly.

Elastic Beanstalk gives you the option to clone an environment. Then, you can deploy your updates and test the new version in the clone. After completing all verification and testing, use the swap URL option in the original environment (blue) to redirect all traffic to your newly deployed version.

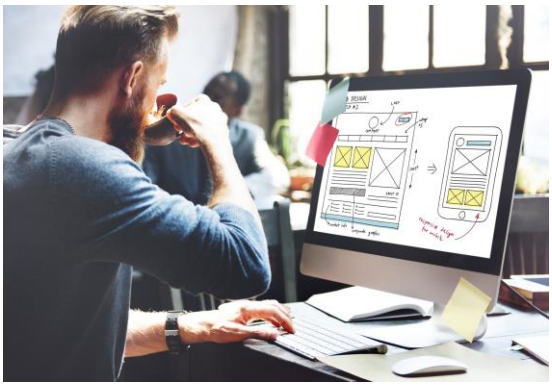
Blue/green deployment is required when you want to update an environment to an incompatible platform version.



The following are the key takeaways from this section of the module:

- You can use Elastics Beanstalk to manage all of the resources that run your application as an environment.
- You can quickly launch a Docker multicontainer environment with Elastic Beanstalk without worrying about Amazon ECS configuration details.
- Deployment options include traffic splitting and blue/green to support testing new versions.

Lab 8.2: Running Containers on a Managed Service



63

You will now complete Lab 8.2: Running Containers on a Managed Service.

Lab: Scenario

- Sofía has containerized the coffee suppliers application, but wants to **reduce the effort to maintain** the application and improve its **scalability**.
- As noted in the previous lab, Sofía wants to move the database to a **managed database service** rather than running it in a container.
- Based on her research, she has made these decisions:
- Use **AWS Elastic Beanstalk** to deploy the application website.
- Use **Amazon Aurora Serverless** for the database. Sofía must retire the container-based MySQL database and load the required user, tables, and data into an Aurora Serverless database.

aws

64

In this lab, you again play the role of Sofía. You will use Elastic Beanstalk and Aurora Serverless to improve the scalability of the coffee supplier application.

Aurora Serverless

Amazon Aurora

Fully managed relational database engine that is compatible with MySQL and PostgreSQL

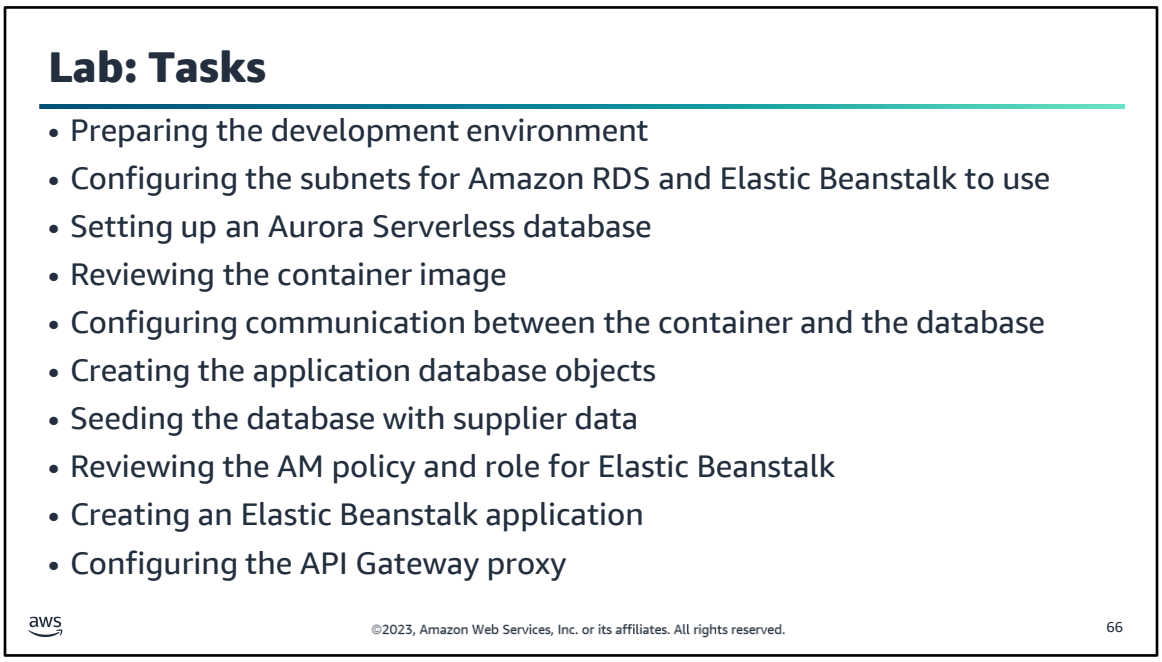
- Part of the Amazon Relational Database Service (Amazon RDS), a managed database service
- Combines the performance and availability of high-end commercial databases with the simplicity and cost-effectiveness of open-source databases
- Offers Aurora Serverless, an on-demand configuration that automatically scales up or down based on traffic and shuts down when not in use

aws

65

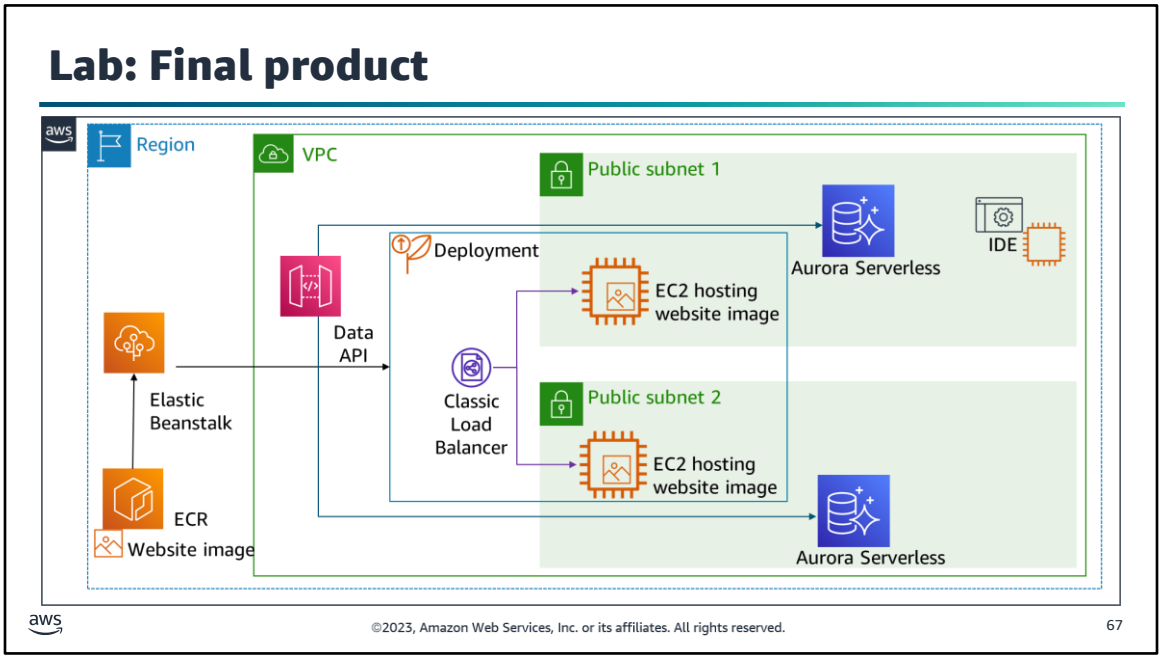
As you learned in the lab scenario, this lab uses Amazon Aurora Serverless. You learned about Amazon Aurora in the AWS Academy Cloud Foundations course. Aurora is a fully managed relational database engine that is compatible with MySQL and PostgreSQL. Aurora is part of the Amazon Relational Database Service (Amazon RDS), a managed database service.

Aurora Serverless is an on-demand automatic scaling configuration for Aurora. An Aurora Serverless database cluster scales compute capacity up and down based on your application's needs. This ability contrasts with Aurora provisioned database clusters, for which you manually manage capacity. Aurora Serverless provides a relatively simple, cost-effective option for infrequent, intermittent, or unpredictable workloads. The service is cost-effective because it automatically starts up, scales compute capacity to match your application's usage, and shuts down when it's not in use.



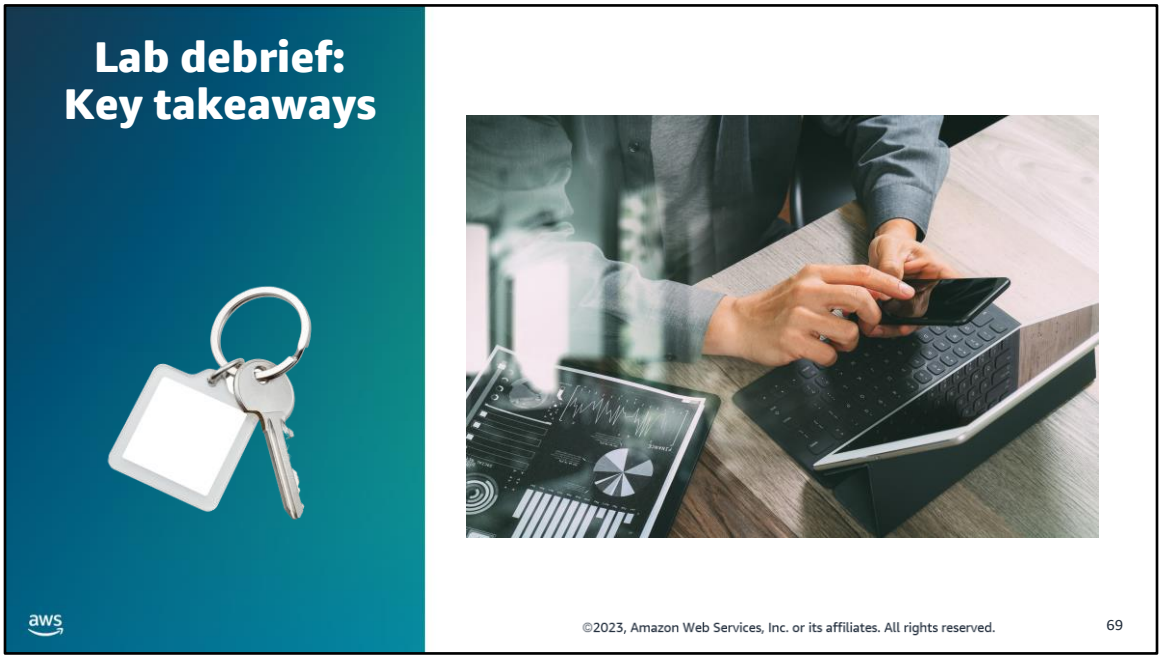
In this lab, you will complete the following tasks:

1. Preparing the development environment
2. Configuring the subnets for Amazon RDS and Elastic Beanstalk to use
3. Setting up an Aurora Serverless database
4. Reviewing the container image
5. Configuring communication between the container and the database
6. Creating the application database objects
7. Seeding the database with supplier data
8. Reviewing the IAM policy and role for Elastic Beanstalk
9. Creating an Elastic Beanstalk application
10. Configuring the Amazon API Gateway proxy



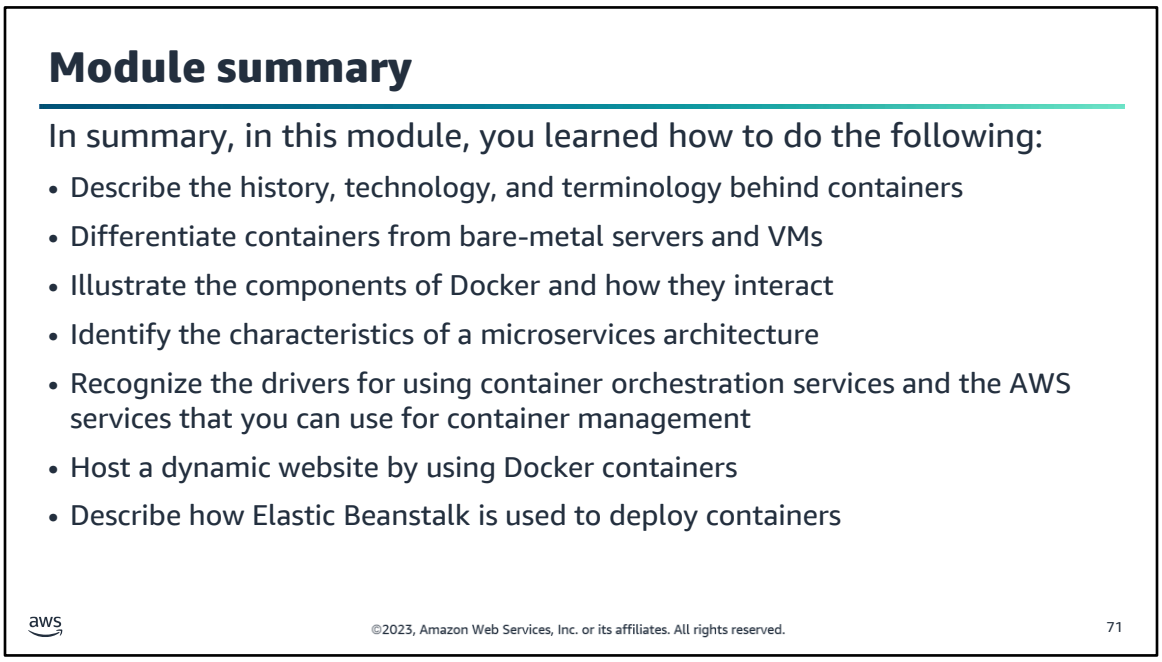
The diagram summarizes what you will have built after you complete the lab.
This architecture shows a VPC with two public subnets across two availability zones. Each availability zone includes an EC2 instance and an Aurora Serverless database. A Classic Load Balancer routes traffic to the two EC2 instances.
API Gateway provides the API endpoint for incoming requests. The diagram shows how Elastic Beanstalk uses a container image stored in Amazon Elastic Container Registry (Amazon ECR) to deploy the containerized website code to the two EC2 instances
The diagram also shows an IDE being used to connect developers to the application.

It is now time to start the lab.



After you have completed the lab, your educator might choose to lead a conversation about the key takeaways from the lab.

It’s now time to review the module and wrap up with a knowledge check and discussion of a practice certification exam question.



In summary, in this module, you learned how to do the following:
- Describe the history, technology, and terminology behind containers
- Differentiate containers from bare-metal servers and VMs
- Illustrate the components of Docker and how they interact
- Identify the characteristics of a microservices architecture
- Recognize the drivers for using container orchestration services and the AWS services that you can use for container management
- Host a dynamic website by using Docker containers
- Describe how Elastic Beanstalk is used to deploy containers

Complete the knowledge check

aws

72

It is now time to complete the knowledge check for this module.

Sample exam question

A cloud architect wants to migrate a web application to containers. The team does not have much experience with AWS or containers, but the architect wants to get them started quickly to be able to experiment.

Which solution would be best?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- **Does not have much experience with AWS or containers**
- **Started quickly**
- **Experiment**

73

It is important to fully understand the scenario and question being asked before even reading the answer choices. Find the keywords in this scenario and question that will help you find the correct answer.

Sample exam question: Response choices

A cloud architect wants to migrate a web application to containers. The team **does not have much experience with AWS or containers**, but the architect wants to get them **started quickly** to be able to **experiment**.

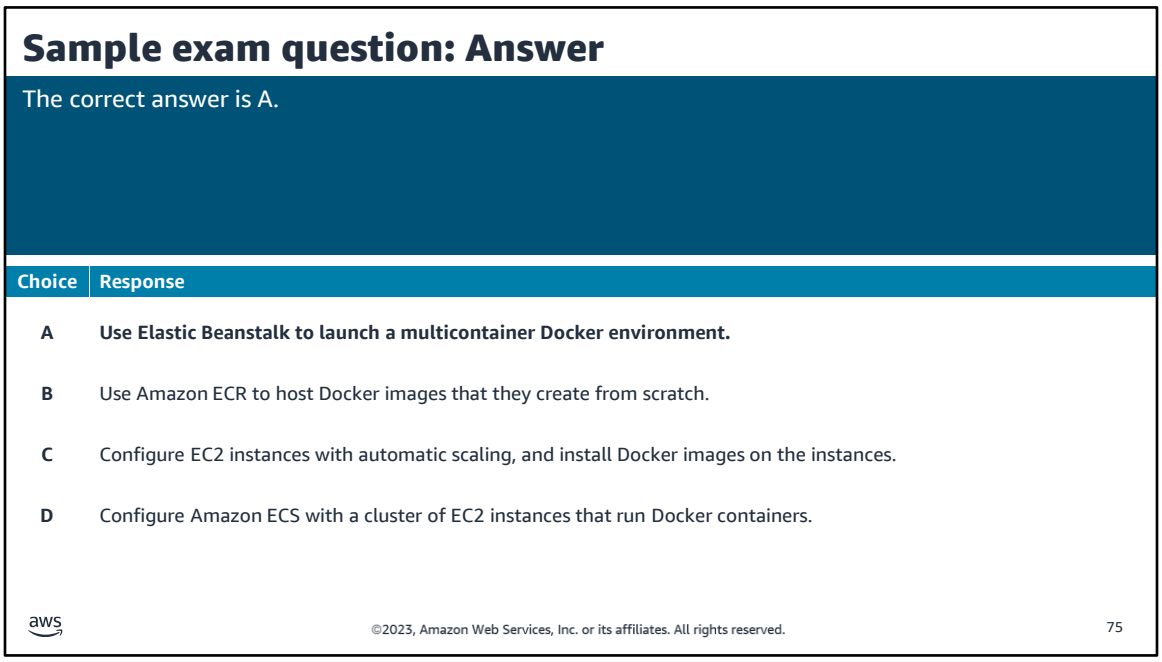
Which solution would be best?

Choice	Response
A	Use Elastic Beanstalk to launch a multicontainer Docker environment.
B	Use Amazon ECR to host Docker images that they create from scratch.
C	Configure EC2 instances with automatic scaling, and install Docker images on the instances.
D	Configure Amazon ECS with a cluster of EC2 instances that run Docker containers.

aws

74

Now that we have bolded the keywords in this scenario, let us look at the answers.

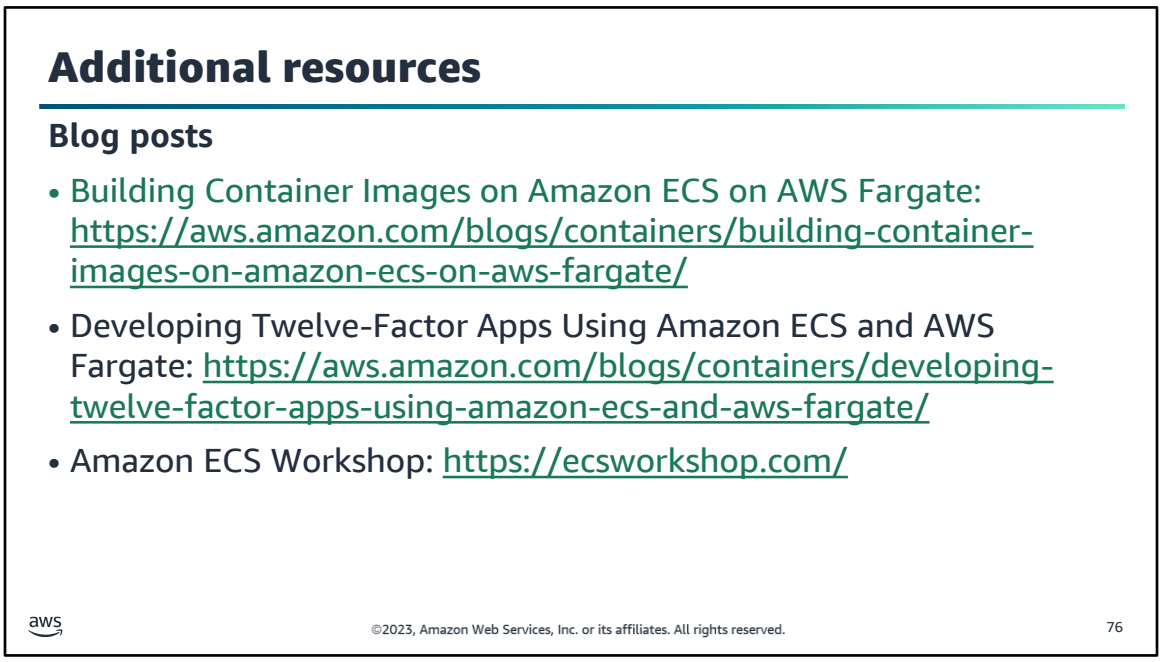


Look at the answer choices, and rule them out based on the keywords that were previously highlighted.

The correct answer is **A**. **Use Elastic Beanstalk to launch a multicontainer Docker environment**.

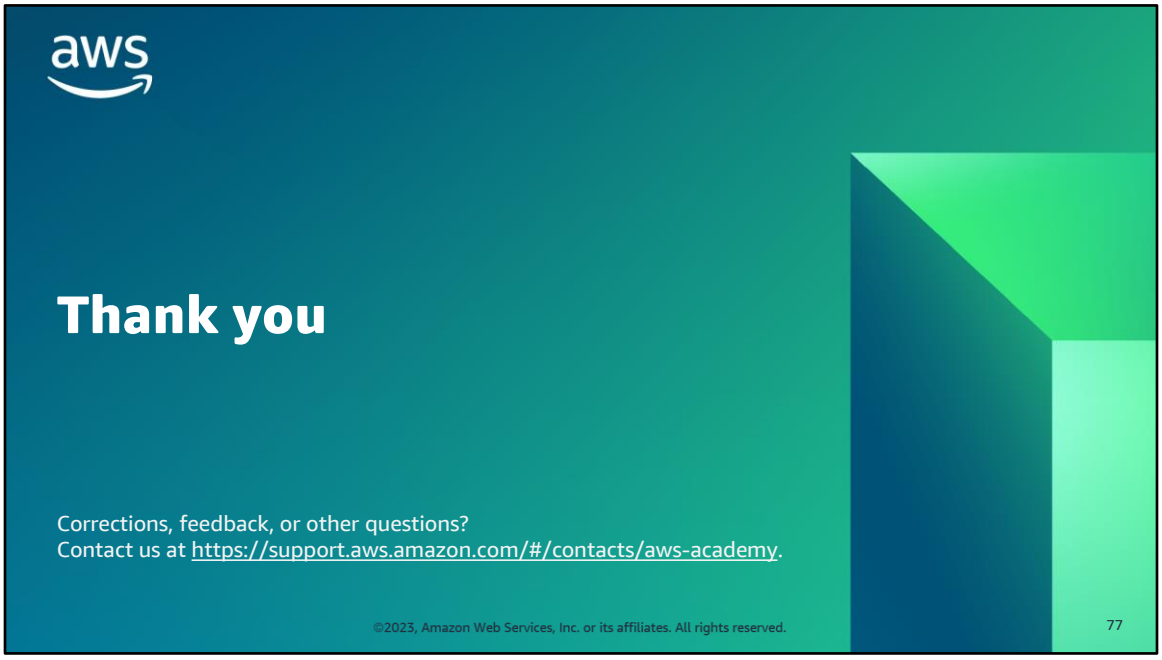
You can use Elastic Beanstalk to set up a working multicontainer Docker environment. You don’t need to deal with the details of configuring instances, networking, and IAM permissions.

You might want to use Amazon ECR, but you would want the team to use existing images as a starting point.



If you want to learn more about the topics covered in this module, you might find the following additional resources helpful:

- Blog posts:
    - Building Container Images on Amazon ECS on AWS Fargate: https://aws.amazon.com/blogs/containers/building-container-images-on-amazon-ecs-on-aws-fargate/.
    - Developing Twelve-Factor Apps Using Amazon ECS and AWS Fargate: https://aws.amazon.com/blogs/containers/developing-twelve-factor-apps-using-amazon-ecs-and-aws-fargate/.
- Amazon ECS Workshop: https://ecsworkshop.com/.



Thank you for completing this module.